

claude-windows / 902e27a6-e026-4b55-8c26-f2c47329cc44

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\902e27a6-e026-4b55-8c26-f2c47329cc44\subagents\workflows\wf\_c287c071-8e9\agent-ab26009d1c2eea60c.jsonl`
- SHA-256: `bf52daa929a477522a071eeb8ce5c29c669a38aed0f03e222a1791478f105d15`
- Source modified: `2026-07-07T14:50:32+00:00`
- Imported at: `2026-07-08T16:00:02+00:00`
- project: `wf\_c287c071-8e9`
- session_id: `902e27a6-e026-4b55-8c26-f2c47329cc44`

Transcript

1. user (2026-07-07T14:45:11.302Z)

You are reviewing an uncommitted change in a git worktree at:
C:/Users/avidu/Projects/khelsutra-guru/bhi-wt-worker-reval
Branch: fix/worker-skip-revalidation-on-dispatch (based on origin/master).

THE BUG BEING FIXED: On the cloud-serving path, the control-plane (backend/orchestration.py) runs validators and ONLY dispatches to the Cloud Run worker on PASS. The worker (backend/worker/__main__.py cmd_infer) then REDUNDANTLY re-ran validation with its OWN config (different min_blur_threshold), rejecting a clip the control-plane already passed. Live failure 2026-07-07: passed at min_blur_threshold=8.0 on the control-plane, worker rejected at its default 20.0 ? GPU dispatch succeeded but the CUJ failed with worker rc=2.

THE FIX (skip-on-dispatch): a new ComputeJobSpec.skip_validation bool (default False) ? backend/compute/cloud_run.py emits a "--skip-validation" worker CLI flag when set ? backend/worker/__main__.py cmd_infer honors it (skips validator_registry.run_all_with_context, sets val_results=[], val_context={}) ? backend/orchestration.py::_maybe_dispatch_remote sets skip_validation=True on every dispatch. The direct "worker infer" CLI (no control-plane) still validates by default. New tests in tests/test_worker.py and tests/test_api_cloud_dispatch.py.

Inspect the ACTUAL code ? do NOT trust this summary. Run: cd "C:/Users/avidu/Projects/khelsutra-guru/bhi-wt-worker-reval" && git --no-pager diff and read the surrounding functions (not just the diff hunks). The full test suite already passes (1779 passed), ruff + mypy are clean, coverage 85.42%. So do NOT report style nits or things the gate already covers. Hunt for REAL correctness/rollout/consumer defects the tests would miss. Only report a finding you can tie to a concrete failure scenario in THIS repo's code.

LENS: DOWNSTREAM CONSUMERS OF EMPTY VALIDATION. When the worker skips validation, val_results=[] and val_context={} on the worker side. Trace whether ANY downstream consumer breaks or silently corrupts:

- The worker's own report.json / intervals.csv (backend/worker/__main__.py _serialize_and_push_outputs, emit_indexer_report, backend/reporting/*). Does empty val_context (no ffprobe_meta) cause a crash or a malformed artifact?
- The control-plane INGEST of the worker result (backend/orchestration.py _ingest_remote_segments, probe_process_cache, cached_process_result, _read_bucket_json). Do any of these READ validation_results or ffprobe_meta from the worker's report.json and rely on them being populated?
- db.add_video with status "processed" + empty results on the worker: any invariant that assumes

validation_results is non-empty for a processed video?
- Is there any code that treats "0 validators ran" as a failure or an error?
Report only real breakages/corruption, with the exact consumer + field.

2. assistant (2026-07-07T14:45:16.560Z)

I'll inspect the actual change and trace the downstream consumers of empty validation.

3. user (2026-07-07T14:45:18.778Z)

```
diff --git a/backend/compute/base.py b/backend/compute/base.py
index 279606e..31af3fe 100644
--- a/backend/compute/base.py
+++ b/backend/compute/base.py
@@ -28,6 +28,13 @@ class ComputeJobSpec:
     segmenter: Optional[str] = None
     # Raw worker ``--set k=v`` overrides. A tuple (not list/dict) so the spec stays
    frozen/hashable.
     config_overrides: tuple[str, ...] = ()
+   # Skip the worker's redundant re-validation. The DISPATCHER (the cloud-serving control-
    plane) is
+   # the authoritative validation gate: it runs the SAME validators with ITS resolved config
    and only
+   # dispatches on PASS, so a worker re-run is (a) redundant, (b) a second full-video decode,
    and
+   # (c) outright WRONG when the worker's config resolves different thresholds ? it can REJECT
    a clip
+   # the gate already passed. Default False keeps the direct ``worker infer`` CLI (no control-
    plane)
+   # validating exactly as before.
+   skip_validation: bool = False
```

```
@dataclass(frozen=True)
diff --git a/backend/compute/cloud_run.py b/backend/compute/cloud_run.py
index 841785d..7247ace 100644
--- a/backend/compute/cloud_run.py
+++ b/backend/compute/cloud_run.py
@@ -196,6 +196,10 @@ class CloudRunCompute(ComputeBackend):
    ]
    if spec.segmenter:
        argv += ["--segmenter", spec.segmenter]
+   if spec.skip_validation:
+       # The control-plane already validated + only dispatches on PASS ? tell the worker
    not to
+       # re-decode and re-validate (redundant, and its config may resolve different
    thresholds).
+       argv.append("--skip-validation")
    for kv in (*spec.config_overrides, *self._container_overrides):
        argv += ["--set", kv]
```

```
diff --git a/backend/orchestration.py b/backend/orchestration.py
index 5e9f510..79b4fed 100644
--- a/backend/orchestration.py
+++ b/backend/orchestration.py
@@ -568,11 +568,18 @@ def _maybe_dispatch_remote(
    prefetch = getattr(wasb_cfg, "prefetch_batches", None)
    if prefetch:
        overrides.append(f"indexing.wasb.prefetch_batches={prefetch}")
+   # Tell the worker to SKIP its redundant re-validation: THIS control-plane is the
    authoritative
+   # gate ? it already ran the validators with its resolved config just above (only a PASS
    reaches
+   # here; a failure returned "failed" before dispatch). A worker re-run would decode the
    video a
```

```

+ # second time and, because its config isn't fully forwarded, can resolve DIFFERENT
thresholds and
+ # REJECT a clip this gate already passed (observed 2026-07-07: passed at
min_blur_threshold=8.0,
+ # then worker-rejected at its default 20.0 after a successful GPU dispatch).
spec = ComputeJobSpec(
    video_uri=req.video_path,
    out_uri=out_root,
    segmenter=seg_name,
    config_overrides=tuple(overrides),
+ skip_validation=True,
)
try:
    # Dispatch the Cloud Run Job + bucket-poll to completion. on_wait is the live
diff --git a/backend/worker/__main__.py b/backend/worker/__main__.py
index 88c50f6..9b19215 100644
--- a/backend/worker/__main__.py
+++ b/backend/worker/__main__.py
@@ -186,6 +186,7 @@ def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
    out_uri=args.out_uri,
    segmenter=args.segmenter,
    config_overrides=tuple(args.set or ()),
+ skip_validation=bool(getattr(args, "skip_validation", False)),
)
try:
    result = compute.run_infer(spec)
@@ -259,10 +260,25 @@ def cmd_infer(args: argparse.Namespace) -> int:
    # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be byte-
identical).
    video_id = compute_video_id(local_video)

- # 3. VALIDATE (same registry as the main CLI).
- val_results, val_context = validator_registry.run_all_with_context(
-     local_video, config
- )
+ # 3. VALIDATE (same registry as the main CLI) ? UNLESS the dispatcher already did.
+ # On the cloud-serving path the CONTROL-PLANE is the authoritative validation gate: it runs
the
+ # SAME validators with ITS resolved config and only dispatches on PASS. Re-validating here
is
+ # redundant, decodes the whole video a SECOND time, and is outright WRONG when the worker's
config
+ # resolves different thresholds than the control-plane's ? observed 2026-07-07: a 5312x2988
clip
+ # (sharpness 11.9) PASSED the control-plane at validation.min_blur_threshold=8.0, then the
worker
+ # REJECTED it at its own default 20.0, so a SUCCESSFUL GPU dispatch failed after the fact.
+ # --skip-validation (set by the dispatcher in orchestration._maybe_dispatch_remote) skips
it; the
+ # direct ``worker infer`` CLI has no control-plane, so it still validates by default.
+ if getattr(args, "skip_validation", False):
+     print(
+         "[worker] validation SKIPPED (--skip-validation): the dispatcher is the
authoritative "
+         "gate and already validated this input."
+     )
+     val_results, val_context = [], {}
+ else:
+     val_results, val_context = validator_registry.run_all_with_context(
+         local_video, config
+     )
    failures = [r for r in val_results if not r.passed]
    db = Database(os.path.join(scratch, config.output.db_name))
    db.add_video(
@@ -724,6 +740,14 @@ def build_parser() -> argparse.ArgumentParser:

```

```

        metavar="k=v",
        help="config override, e.g. --set indexing.skip_ai_handoff=true",
    )
+   pi.add_argument(
+       "--skip-validation",
+       action="store_true",
+       help="skip the worker's re-validation: the dispatcher (control-plane) already ran the "
+       "validators with its resolved config and only dispatches on PASS, so re-validating here "
+       "is "
+       "redundant, decodes the video again, and can contradict the gate. Default-OFF: the "
+       "direct "
+       "`worker infer` CLI still validates.",
+   )
+   pi.set_defaults(func=cmd_infer)

    pt = sub.add_parser(
diff --git a/tests/test_api_cloud_dispatch.py b/tests/test_api_cloud_dispatch.py
index 8382234..19c37d9 100644
--- a/tests/test_api_cloud_dispatch.py
+++ b/tests/test_api_cloud_dispatch.py
@@ -236,6 +236,22 @@ def test_cloud_dispatch_omits_unset_wasb_tuning(cloud_env):
    assert "indexing.wasb.prefetch_batches" not in joined

+def test_cloud_dispatch_skips_worker_revalidation(cloud_env):
+    """The control-plane is the AUTHORITATIVE validation gate: it validated + only dispatches
+    on PASS
+    (registry.run_all_with_context is stubbed to pass in cloud_env). Re-validating on the
+    worker is
+    redundant, decodes the video a second time, and ? because the control-plane's validation
+    config
+    isn't forwarded ? can REJECT a clip the gate already passed (observed 2026-07-07: passed at
+    validation.min_blur_threshold=8.0, then worker-rejected at its default 20.0). So the
+    dispatch MUST
+    carry --skip-validation, telling the worker to trust the gate. Regression guard for that
+    wiring."""
+    _db, _bucket, monkeypatch = cloud_env
+    fake = _FakeCloudClient()
+    _inject_fake(monkeypatch, fake)
+    m._execute_processing(
+        m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
+    )
+    assert "--skip-validation" in fake.calls[0]
+
+def test_cloud_failure_marks_failed_no_rows(cloud_env):
+    db, _bucket, monkeypatch = cloud_env
+    # pre-seed a prior good row ? must be preserved on a cloud failure (destroy-AFTER-confirm).
diff --git a/tests/test_worker.py b/tests/test_worker.py
index 8fb5e16..72b2514 100644
--- a/tests/test_worker.py
+++ b/tests/test_worker.py
@@ -97,6 +97,14 @@ def test_parser_infer_accepts_set_and_uris():
    assert args.set == ["indexing.skip_ai_handoff=true", "sport=tennis"]

+def test_parser_infer_skip_validation_flag():
+    base = ["infer", "--video-uri", "s3://b/v.mp4", "--out-uri", "s3://b"]
+    # Default-OFF: the direct `worker infer` CLI still validates.
+    assert wmain.build_parser().parse_args(base).skip_validation is False
+    # The dispatcher passes it to disable the redundant worker re-validation.
+    assert wmain.build_parser().parse_args(base + ["--skip-validation"]).skip_validation is
+    True
+
+
+

```

```
# ----- cmd_infer (local, mocked)

@@ -407,6 +415,43 @@ def test_cmd_infer_validation_failure_returns_2(tmp_path, monkeypatch):
    assert marker["returncode"] == 2 and marker["status"] == "failed"

+def test_cmd_infer_skip_validation_bypasses_validators(tmp_path, monkeypatch):
+    """--skip-validation (set by the dispatcher, the authoritative gate) must NOT run the
+    worker's
+    validators ? no second decode, and no chance of a threshold mismatch REJECTING a clip the
+    control-plane already passed (the live 2026-07-07 CUJ failure). Prove it by making
+    run_all_with_context RAISE: a skipped run never touches it, so the job completes rc 0 with
+    a
+    status="success" report. Contrast test_cmd_infer_validation_failure_returns_2 (no flag ? rc
+    2)."""
+    monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidSkip")
+
+    def _boom(*a, **k): # the validators must never be reached on the skip path
+        raise AssertionError("run_all_with_context must not run under --skip-validation")
+
+    monkeypatch.setattr(wmain.validator_registry, "run_all_with_context", _boom)
+    monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _FakeSeg)
+    monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
+    video = tmp_path / "in.mp4"
+    video.write_bytes(b"FAKE")
+    out = tmp_path / "bucket"
+
+    rc = wmain.main(
+        [
+            "infer",
+            "--video-uri",
+            str(video),
+            "--out-uri",
+            str(out),
+            "--scratch",
+            str(tmp_path / "s"),
+            "--segmenter",
+            "wasb_hybrid",
+            "--skip-validation",
+        ]
+    )
+    assert rc == 0 # a clip that WOULD fail re-validation still succeeds ? the gate is trusted
+    report = json.loads((out / "outputs" / "vidSkip" / "report.json").read_text())
+    assert report["status"] == "success" and report["segment_count"] == 2
+
+    # --- cmd_train: pull labels -> synth trajectories -> manifest -> harness -> push bundle ---
```

4. assistant (2026-07-07T14:45:24.937Z)

Let me read the key files to trace downstream consumers of empty validation.

5. user (2026-07-07T14:45:26.485Z)

```
1     """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3     python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4         [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5     python -m backend.worker train ...          # Gen-0 train-from-bucket (follow-up)
6
7     The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
8     ``local://``
9     is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
10    local runs
```

```

9     are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
10    ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
key) ?
11    the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
12
13    This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
CLI uses
14    (validator registry ? segmenter ? report), never a forked pipeline.
15    ""
16
17    from __future__ import annotations
18
19    import argparse
20    import csv
21    import json
22    import logging
23    import os
24    import re
25    import sys
26    import tempfile
27    from typing import Any, List
28
29    from backend import storage
30    from backend.compute import ComputeJobSpec, get_compute_backend
31    from backend.config import (
32        StartupCheckError,
33        enforce_startup_checks,
34        load_config,
35        probe_cuda_available,
36    )
37    from backend.infrastructure.database import Database
38    from backend.pipeline.segmenters.base import segmenter_registry
39    from backend.pipeline.validators.base import registry as validator_registry
40    from backend.reporting import emit_indexer_report
41    from backend.results import Failure
42    from backend.utils.ffmpeg import ffprobe_bin # P2a: single ffmpeg/ffprobe resolver
43    from backend.utils.hashing import compute_video_id
44
45    logger = logging.getLogger("backend.worker")
46
47
48    def _coerce(v: str) -> Any:
49        """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
str."""
50        low = v.lower()
51        if low in ("true", "false"):
52            return low == "true"
53        for cast in (int, float):
54            try:
55                return cast(v)
56            except ValueError:
57                pass
58        return v
59
60
61    def _apply_overrides(config: Any, sets: List[str]) -> None:
62        """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
63        for kv in sets or []:
64            key, sep, val = kv.partition("=")
65            if not sep:
66                raise ValueError(f"--set expects k=v, got {kv!r}")
67            parts = key.split(".")
68            obj = config
69            for p in parts[:-1]:

```

```

70         obj = getattr(obj, p)
71         setattr(obj, parts[-1], _coerce(val))
72
73
74     def _write_intervals_csv(segments: List[dict], path: str) -> None:
75         """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
friendly
76         artifact (same schema as the rally-annotator labels)."""
77         with open(path, "w", newline="") as f:
78             w = csv.writer(f)
79             w.writerow(["start", "end", "shot_count", "ending_reason"])
80             for s in segments:
81                 w.writerow(
82                     [
83                         f"{float(s.get('start_time', 0.0)):.3f}",
84                         f"{float(s.get('end_time', 0.0)):.3f}",
85                         s.get("shot_count", ""),
86                         s.get("ending_reason", s.get("shot_count_confidence", "")),
87                     ]
88                 )
89
90
91     def _write_report_json(
92         video_id: str, segments: List[dict], status: str, path: str, video_uri: str = ""
93     ) -> None:
94         with open(path, "w") as f:
95             json.dump(
96                 {
97                     "video_id": video_id,
98                     "status": status,
99                     "segment_count": len(segments),
100                     # The SOURCE the detection ran on (CP-rebuild P3, #485): a restart-wiped
101                     # control plane re-hydrating from outputs/<md5>/ restores the video row
102                     # with its truthful path ? without this, only the segments recover.
103                     "video_uri": video_uri,
104                     "segments": [
105                         {
106                             "start": float(s.get("start_time", 0.0)),
107                             "end": float(s.get("end_time", 0.0)),
108                         }
109                         for s in segments
110                     ],
111                 },
112                 f,
113                 indent=2,
114             )
115
116
117     def _serialize_and_push_outputs(
118         scratch: str,
119         out_uri: str,
120         video_id: str,
121         segments: List[dict],
122         status: str,
123         storage_cfg: dict,
124         video_uri: str = "",
125     ) -> List[str]:
126         """Write intervals.csv + report.json under ``<scratch>/outputs/<video_id>/`` and
push both to
127         ``<out_uri>/outputs/<video_id>/``; return the pushed URIs.
128
129         Shared by the SUCCESS path and the failure path (``_fail``) so a detector
timeout/abort leaves a
130         ``status="failed"`` report.json in the bucket ? the control plane's source of truth
131         (``orchestration.cached_process_result`` / ``probe_process_cache`` treat any

```

```

non-``success``
132     report as "no usable result" ? retry, and the restart-recovery poll waits for
report.json to
133     EXIST) ? instead of a missing artifact or a false 0-segment ``"success"``. ""
134     out_local = os.path.join(scratch, "outputs", video_id)
135     os.makedirs(out_local, exist_ok=True)
136     _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
137     _write_report_json(
138         video_id,
139         segments,
140         status,
141         os.path.join(out_local, "report.json"),
142         video_uri=video_uri,
143     )
144     out_prefix = out_uri.rstrip("/")
145     pushed: List[str] = []
146     for fn in sorted(os.listdir(out_local)):
147         uri = f"{out_prefix}/outputs/{video_id}/{fn}"
148         storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
149         pushed.append(uri)
150     return pushed
151
152
153     def _run_startup_checks(config: Any) -> bool:
154         """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
155         best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
156         cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
no-op
157         (returns True, ZERO work) when no deployment.profile is selected.
158
159         The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily
imports
160         torch, and pre-M5 the worker's infer/train path was torch-free (detection is
delegated to a
161         WSL/native subprocess). Probing only when a profile is set keeps the default-OFF
path a true
162         no-op of work, not just of output."""
163         profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
164         cuda = (
165             probe_cuda_available() if profile else None
166         ) # torch-free on the default-OFF path
167         try:
168             enforce_startup_checks(config, cuda_available=cuda, logger=logger)
169             return True
170         except StartupCheckError:
171             return False # already logged at ERROR by enforce_startup_checks
172
173
174     def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
175         """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc
once the
176         outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
177         container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
re-dispatch.
178
179         A remote job that never writes a completion marker (a failed/hung container) makes
the bucket
180         poll exhaust its budget ? ``PolicyTimeout``. Catch it and return a CLEAN rc 4
(remote dispatch
181         timeout/failure, distinct from the local 2/3) rather than letting a raw traceback
escape."""
182         from backend.infrastructure.execution_policy import PolicyTimeout

```



```

183
184     spec = ComputeJobSpec(
185         video_uri=args.video_uri,
186         out_uri=args.out_uri,
187         segmenter=args.segmenter,
188         config_overrides=tuple(args.set or ()),
189         skip_validation=bool(getattr(args, "skip_validation", False)),
190     )
191     try:
192         result = compute.run_infer(spec)
193     except PolicyTimeout as e:
194         logger.warning(
195             "[worker] remote compute did not complete (no marker within budget): %s", e
196         )
197         return 4
198     logger.info(
199         "[worker] remote compute done: video_id=%s rc=%d outputs=%s",
200         result.video_id,
201         result.returncode,
202         result.outputs_uri,
203     )
204     return result.returncode
205
206
207 def _write_done_marker(
208     done_uri: str,
209     video_id: str,
210     returncode: int,
211     segment_count: int,
212     status: str,
213     storage_cfg: dict,
214     scratch: str,
215 ) -> None:
216     """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
for a remote
217     dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
return."""
218     try:
219         marker = {
220             "video_id": video_id,
221             "returncode": returncode,
222             "segment_count": segment_count,
223             "status": status,
224         }
225         local = os.path.join(scratch, "_done.json")
226         with open(local, "w", encoding="utf-8") as f:
227             json.dump(marker, f)
228         storage.put(local, done_uri, config=storage_cfg)
229         logger.info("[worker] wrote completion marker -> %s", done_uri)
230     except Exception as e: # never fail a good run because the marker push hiccuped
231         logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)
232
233
234 def cmd_infer(args: argparse.Namespace) -> int:
235     config = load_config(args.config) if args.config else load_config()
236     _apply_overrides(config, args.set)
237     if not _run_startup_checks(config):
238         return 2
239
240     # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
process.
241     # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
through to the
242     # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
(the

```

```

243         # dispatcher forces compute_target=local_gpu), so it never recursively re-
dispatches.
244         compute = get_compute_backend(config)
245         if compute is not None:
246             return _dispatch_remote(compute, args)
247
248         storage_cfg = config.storage.model_dump()
249
250         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
251         os.makedirs(scratch, exist_ok=True)
252         config.output.output_dir = scratch
253
254         # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
255         local_video = storage.fetch(
256             args.video_uri, os.path.join(scratch, "in.mp4"), config=storage_cfg
257         )
258         print(f"[worker] pulled {args.video_uri} -> {local_video}")
259
260         # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
261         video_id = compute_video_id(local_video)
262
263         # 3. VALIDATE (same registry as the main CLI) ? UNLESS the dispatcher already did.
264         # On the cloud-serving path the CONTROL-PLANE is the authoritative validation gate:
it runs the
265         # SAME validators with ITS resolved config and only dispatches on PASS. Re-
validating here is
266         # redundant, decodes the whole video a SECOND time, and is outright WRONG when the
worker's config
267         # resolves different thresholds than the control-plane's ? observed 2026-07-07: a
5312x2988 clip
268         # (sharpness 11.9) PASSED the control-plane at validation.min_blur_threshold=8.0,
then the worker
269         # REJECTED it at its own default 20.0, so a SUCCESSFUL GPU dispatch failed after the
fact.
270         # --skip-validation (set by the dispatcher in orchestration._maybe_dispatch_remote)
skips it; the
271         # direct ``worker infer`` CLI has no control-plane, so it still validates by
default.
272         if getattr(args, "skip_validation", False):
273             print(
274                 "[worker] validation SKIPPED (--skip-validation): the dispatcher is the
authoritative "
275                 "gate and already validated this input."
276             )
277             val_results, val_context = [], {}
278         else:
279             val_results, val_context = validator_registry.run_all_with_context(
280                 local_video, config
281             )
282         failures = [r for r in val_results if not r.passed]
283         db = Database(os.path.join(scratch, config.output.db_name))
284         db.add_video(
285             video_id,
286             local_video,
287             "failed" if failures else "processed",
288             [r.model_dump() for r in val_results],
289         )
290
291         def _fail(rc: int) -> int:
292             # A worker-side FAILURE (validation, unknown segmenter, or a detector
timeout/abort the
293             # segmenter surfaced as a Failure) must NOT masquerade as a 0-rally "success".
Write + push

```

```

294         # a status="failed" report.json/intervals.csv (0 segments) so the control
plane's bucket
295         # truth (submit-cache probe, restart-recovery) and the alpha-user result surface
see an
296         # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup
must never
297         # mask the original failure. THEN the M6 done-marker (real rc + status) so a
remote
298         # dispatcher's bucket-poll terminates FAST instead of hanging until its budget
expires.
299         try:
300             _serialize_and_push_outputs(
301                 scratch,
302                 args.out_uri,
303                 video_id,
304                 [],
305                 "failed",
306                 storage_cfg,
307                 str(getattr(args, "video_uri", "") or ""),
308             )
309         except Exception as e: # never mask the real failure on an artifact-push hiccup
310             logger.warning("[worker] could not push failure artifacts: %s", e)
311         # DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
312         if getattr(args, "done_uri", None):
313             _write_done_marker(
314                 args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
315             )
316         return rc
317
318     segments: List[dict] = []
319     segmenter_actual = None
320     segmentation_ran = False
321     status = "failed"
322     try:
323         if failures:
324             print(
325                 "[worker] validation FAILED: "
326                 + "; ".join(f"{r.validator_name}: {r.message}" for r in failures)
327             )
328             return _fail(2)
329         seg_name = args.segmenter or config.indexing.default_segmenter
330         segmenter_cls = segmenter_registry.get(seg_name)
331         if not segmenter_cls:
332             print(
333                 f"[worker] unknown segmenter: {seg_name!r} "
334                 f"(have {list(segmenter_registry.list_available())})"
335             )
336             return _fail(2)
337         segmenter_actual = seg_name
338         result = segmenter_cls(db, config).process_video(
339             video_id, local_video, max_frames=args.max_frames
340         )
341         segmentation_ran = True
342         if isinstance(result, Failure):
343             print(f"[worker] segmenter FAILED: {result.message}")
344             return _fail(3)
345         segments = result.value if hasattr(result, "value") else result
346         status = "success"
347         # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
348         # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
349         # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
350         if not segments:

```

```

351         detector_impl = (
352             os.environ.get("RALLY_DETECTOR_IMPL")
353             or getattr(config.indexing, "detector_impl", None)
354             or "auto"
355         )
356         logger.warning(
357             "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
358             "The detector substrate yielded no usable trajectories/windows ? check
the detector "
359             "logs above; common causes: a missing ${AI_PROVIDER}_API_KEY with "
360             "skip_ai_handoff=False (the segmenter bails BEFORE detection ? set the
key or "
361             "indexing.skip_ai_handoff=true; cloud-serving now fails this at
startup), native "
362             "runner UNAVAILABLE (weights/repo/WASB_PYTHON env), the WASB env
produced no "
363             "detections, or the input resolution differs from the tuned proxy
resolution. "
364             "Re-run with -v for the native command + frame counts.",
365             video_id,
366             segmenter_actual,
367             detector_impl,
368         )
369     finally:
370         # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
371         emit_indexer_report(
372             db=db,
373             video_id=video_id,
374             video_path=local_video,
375             config=config,
376             val_results=val_results,
377             val_context=val_context,
378             segmenter_requested=args.segmenter,
379             segmenter_actual=segmenter_actual,
380             was_reingest=False,
381             segmentation_ran=segmentation_ran,
382         )
383
384     # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
385     pushed = _serialize_and_push_outputs(
386         scratch,
387         args.out_uri,
388         video_id,
389         segments,
390         status,
391         storage_cfg,
392         str(getattr(args, "video_uri", "") or ""),
393     )
394
395     print(
396         f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):"
397     )
398     for u in pushed:
399         print("    ", u)
400
401     # M6: a remote dispatcher passes --done-uri; write the SUCCESS completion marker so
its bucket-poll
402     # terminates (carries video_id ? the dispatcher's output path + rc=0). Worker
FAILURES (rc 2/3)
403     # write their own marker via _fail() above, so the dispatcher fails FAST either way
(no 30-min hang
404     # on a bad video). DEFAULT-OFF: no --done-uri ? no marker written (the unchanged

```

```

local run).
405     if getattr(args, "done_uri", None):
406         _write_done_marker(
407             args.done_uri, video_id, 0, len(segments), status, storage_cfg, scratch
408         )
409     return 0
410
411
412     #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
413     #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
414     _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")
415
416     #: Leading ISO-date prefix on a canonical label name (`labels/<YYYY-MM-
DD>_<name>.rallies.csv`,
417     #: docs/DATA_IN_GCS.md §7.1). It disambiguates recycled GoPro names by authoring date
but is NOT
418     #: part of the clip's join key, so it's stripped when deriving the videos/ stem.
419     _LABEL_DATE_PREFIX = re.compile(r"^\d{4}-\d{2}-\d{2}_")
420
421
422     def _label_clip_name(fname: str) -> str:
423         """Derive the clip name (the ``videos/<name>.<ext>`` join key) from a label
filename.
424
425         Strips both the ``.rallies.csv`` / ``.csv`` suffix and an optional leading ISO-date
prefix, so a
426         canonical dated label and a legacy bare label resolve to the SAME bare clip stem:
427
428         * ``2026-06-22_GX020094.rallies.csv`` -> ``GX020094`` (canonical, date-
stamped)
429         * ``GX020094.rallies.csv`` -> ``GX020094`` (legacy bare ? same
stem)
430         * ``GX020094_proxy.rallies.csv`` -> ``GX020094_proxy`` (``_proxy`` is part of
the name, kept)
431         * ``2026-06-21_mahadevpura_1.rallies.csv`` -> ``mahadevpura_1`` (underscores in
<name> survive)
432
433         Without the date strip, ``--trajectory-source detector`` can't match a dated label
to its bare
434         video stem, so every clip is skipped and train aborts with ``<2 videos``
(DATA_IN_GCS §7b #1).
435         """
436         name = fname.replace(".rallies.csv", "").replace(".csv", "")
437         return _LABEL_DATE_PREFIX.sub("", name)
438
439
440     def _probe_video_fps_width(path: str):
441         """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.
442
443         The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
WRONG
444         fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
445         cv2's ``_probe_fps_width`` silently defaults to (30, 1280) on a read miss ? fine for
the
446         inference report (it flags the fallback) but corrupting for REAL training data. So
here we
447         probe with ffprobe (same tool as ``get_video_duration``) and return None so the
caller SKIPS
448         rather than guesses.
449         """
450         import json
451         import subprocess

```

```

452
453     try:
454         out = subprocess.check_output(
455             [
456                 ffprobe_bin(),
457                 "-v",
458                 "error",
459                 "-select_streams",
460                 "v:0",
461                 "-show_entries",
462                 "stream=r_frame_rate,width",
463                 "-of",
464                 "json",
465                 path,
466             ],
467             stderr=subprocess.DEVNULL,
468             ).decode()
469         st = (json.loads(out).get("streams") or [{}])[0]
470         num, _ = str(st.get("r_frame_rate", "0/1")).partition("/")
471         fps = float(num) / float(den or "1")
472         width = float(st.get("width") or 0)
473         if fps > 0 and width > 0:
474             return fps, width
475     except (
476         subprocess.SubprocessError,
477         ValueError,
478         KeyError,
479         OSError,
480         json.JSONDecodeError,
481     ):
482         pass
483     return None
484
485
486 def _resolve_train_detector(args: argparse.Namespace, config: Any):
487     """Build the trajectory DetectorRunner for `--trajectory-source detector`, honouring
488     detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
489     box,
490     stub=CPU mock). Returns the runner, or prints an error + returns None.
491
492     Reuses the segmenter's `_resolve_runner` seam so the worker and the live CLI build
493     the
494     detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
495     has
496     no shuttle detector and is rejected.
497     """
498     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
499
500     seg_cls = segmenter_registry.get(args.segmenter)
501     if not seg_cls:
502         print(
503             f"[worker] unknown segmenter: {args.segmenter!r} "
504             f"(have {list(segmenter_registry.list_available())})"
505         )
506         return None
507     seg = seg_cls(None, config)
508     if not isinstance(seg, TrajectoryHybridSegmenter):
509         print(
510             f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
511             f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid / fusion_hybrid)."
512         )
513         return None
514
515     try:
516         runner, _ = seg._resolve_runner(config.indexing)
517     except NotImplementedError as e:

```

```

514         # e.g. detector_impl='native' for a family without a native runner (tracknet, or
515         # fusion with a tracknet substrate). Fail cleanly (rc!=0), not with a raw
traceback.
516         print(f"[worker] detector not available: {e}")
517         return None
518     ok, msg = runner.healthcheck()
519     if not ok:
520         print(f"[worker] detector environment not ready: {msg}")
521         return None
522     print(f"[worker] detector ready ({args.segmenter}): {msg}")
523     return runner
524
525
526     def cmd_train(args: argparse.Namespace) -> int:
527         """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
shell out
528         to training.gen0.harness (backend must NOT import training) -> push the artifact
bundle.
529
530         Two trajectory sources (the train pipeline + harness are identical either way):
531         * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
synthetic
532         shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
CI.
533         * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
534         DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
This
535         is the M3.5 "first real training" path; only the trajectory source flips.
536         """
537         import subprocess
538
539         config = load_config(args.config) if args.config else load_config()
540         _apply_overrides(config, args.set)
541         if not _run_startup_checks(config):
542             return 2
543         storage_cfg = config.storage.model_dump()
544
545         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
546         labels_dir = os.path.join(scratch, "labels")
547         traj_dir = os.path.join(scratch, "trajectories")
548         videos_dir = os.path.join(scratch, "videos")
549         os.makedirs(labels_dir, exist_ok=True)
550         os.makedirs(traj_dir, exist_ok=True)
551
552         corpus = args.corpus_uri.rstrip("/")
553         label_uris = sorted(
554             u
555             for u in storage.list_uris(f"{corpus}/labels/", config=storage_cfg)
556             if u.lower().endswith(".csv")
557         )
558         if len(label_uris) < 2:
559             print(
560                 f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
561                 f"under {corpus}/labels/"
562             )
563             return 2
564
565         # Real-detector source: resolve the runner once + index the bucket's videos/ by
stem.
566         runner = None
567         video_by_stem: dict = {}
568         if args.trajectory_source == "detector":
569             os.makedirs(videos_dir, exist_ok=True)
570             runner = _resolve_train_detector(args, config)

```

```

571         if runner is None:
572             return 2
573         for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
574             vname = storage.parse_uri(vuri).name
575             if not vname.lower().endswith(_VIDEO_EXTS):
576                 continue # skip sidecars (.srt/.json/thumbnails) so they can't shadow
the video
577                 video_by_stem[os.path.splitext(vname)[0]] = vuri
578
579     print(f"[worker] trajectory source: {args.trajectory_source}")
580     specs: List[dict] = []
581     for uri in label_uris:
582         fname = storage.parse_uri(uri).name # e.g. 2026-06-22_GX020094.rallies.csv
583         name = _label_clip_name(fname) # -> GX020094 (date prefix stripped)
584         local_label = storage.fetch(
585             uri, os.path.join(labels_dir, fname), config=storage_cfg
586         )
587         if args.trajectory_source == "detector":
588             vuri = video_by_stem.get(name)
589             if not vuri:
590                 print(
591                     f"[worker] no video for {name!r} under {corpus}/videos/ ? skipping."
592                 )
593                 continue
594             local_video = storage.fetch(
595                 vuri,
596                 os.path.join(videos_dir, storage.parse_uri(vuri).name),
597                 config=storage_cfg,
598             )
599             video_id = compute_video_id(local_video)
600             # Real fps/width drive the trajectory frame->time mapping; PROBE (don't
guess) ? a
601             # wrong fps silently mis-aligns every label, so skip the video if ffprobe
can't read it.
602             meta = _probe_video_fps_width(local_video)
603             if meta is None:
604                 print(
605                     f"[worker] could not probe fps/width for {name!r} (ffprobe) ?
skipping (won't guess)."
606                 )
607                 continue
608             fps, frame_width = meta
609             traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
610             if not traj:
611                 print(
612                     f"[worker] detector produced no trajectory for {name!r} ? skipping."
613                 )
614                 continue
615             specs.append(
616                 {
617                     "name": name,
618                     "trajectory": traj,
619                     "labels": local_label,
620                     "fps": fps,
621                     "frame_width": frame_width,
622                 }
623             )
624         else:
625             from backend.pipeline.detectors.stub_runner import (
626                 synth_trajectory_from_labels,
627             )
628
629             traj = os.path.join(traj_dir, f"{name}_traj.csv")
630             synth_trajectory_from_labels(
631                 local_label, traj, fps=args.fps, frame_width=args.frame_width

```



```

632         )
633         specs.append(
634             {
635                 "name": name,
636                 "trajectory": traj,
637                 "labels": local_label,
638                 "fps": args.fps,
639                 "frame_width": args.frame_width,
640             }
641         )
642
643     if len(specs) < 2:
644         print(
645             f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}."
646         )
647         return 2
648     manifest_path = os.path.join(scratch, "manifest.json")
649     with open(manifest_path, "w", encoding="utf-8") as f:
650         json.dump(specs, f, indent=2)
651     src_label = (
652         "real detector" if args.trajectory_source == "detector" else "CPU-mock stub"
653     )
654     print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")
655
656     out_dir = os.path.join(scratch, "artifacts")
657     cmd = [
658         sys.executable,
659         "-m",
660         "training.gen0.harness",
661         "--manifest",
662         manifest_path,
663         "--out",
664         out_dir,
665         "--version",
666         args.version,
667     ]
668     if args.floor:
669         floor_local = (
670             storage.fetch(
671                 args.floor, os.path.join(scratch, "floor.json"), config=storage_cfg
672             )
673             if "://" in args.floor
674             else args.floor
675         )
676         cmd += ["--floor", floor_local]
677     print("[worker] training:", " ".join(cmd))
678     rc = subprocess.run(cmd).returncode
679     if rc != 0:
680         print(f"[worker] gen0 harness failed (rc={rc})")
681         return rc
682
683     # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
684     bundle = os.path.join(out_dir, args.version)
685     out_prefix = args.out_uri.rstrip("/")
686     pushed = []
687     for fn in sorted(os.listdir(bundle)):
688         uri = f"{out_prefix}/weights/{args.version}/{fn}"
689         storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
690         pushed.append(uri)
691     print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
692     for u in pushed:
693         print("    ", u)
694     return 0
695

```

```

696
697 def build_parser() -> argparse.ArgumentParser:
698     p = argparse.ArgumentParser(
699         prog="python -m backend.worker",
700         description="Storage-aware worker: pull ? run pipeline ? push.",
701     )
702     p.add_argument(
703         "-v",
704         "--verbose",
705         action="store_true",
706         help="DEBUG logging (the native detector command, frame counts, per-stage
detail)",
707     )
708     sub = p.add_subparsers(dest="cmd", required=True)
709
710     pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
711     pi.add_argument(
712         "--video-uri", required=True, help="local path / local:// / s3:// / gcs://"
713     )
714     pi.add_argument(
715         "--out-uri",
716         required=True,
717         help="output prefix (outputs/<video_id>/? is appended)",
718     )
719     pi.add_argument(
720         "--segmenter", default=None, help="default: indexing.default_segmenter"
721     )
722     pi.add_argument(
723         "--config", default=None, help="config.json path (default: ./config.json)"
724     )
725     pi.add_argument(
726         "--scratch", default=None, help="local scratch dir (default: a temp dir)"
727     )
728     pi.add_argument("--max-frames", type=int, default=None)
729     pi.add_argument(
730         "--done-uri",
731         default=None,
732         help="M6: storage URI for a completion MARKER (video_id + rc) written on a "
733         "successful run, so a remote (Cloud Run) dispatcher's bucket-poll terminates. "
734         "Default-OFF: omit it for the normal local run.",
735     )
736     pi.add_argument(
737         "--set",
738         action="append",
739         default=[],
740         metavar="k=v",
741         help="config override, e.g. --set indexing.skip_ai_handoff=true",
742     )
743     pi.add_argument(
744         "--skip-validation",
745         action="store_true",
746         help="skip the worker's re-validation: the dispatcher (control-plane) already
ran the "
747         "validators with its resolved config and only dispatches on PASS, so re-
validating here is "
748         "redundant, decodes the video again, and can contradict the gate. Default-OFF:
the direct "
749         "`worker infer` CLI still validates.",
750     )
751     pi.set_defaults(func=cmd_infer)
752
753     pt = sub.add_parser(
754         "train",
755         help="Gen-0 train-from-bucket (labels [+videos] -> trajectories -> harness)",
756     )

```

```

757     pt.add_argument(
758         "--corpus-uri",
759         required=True,
760         help="prefix containing labels/ (+ videos/ for "
761         "--trajectory-source detector), e.g. s3://bucket or gcs://bucket",
762     )
763     pt.add_argument(
764         "--out-uri",
765         required=True,
766         help="output prefix (weights/<version>/ is appended)",
767     )
768     pt.add_argument("--version", required=True, help="semver for the artifact bundle")
769     pt.add_argument(
770         "--trajectory-source",
771         choices=["synth", "detector"],
772         default="synth",
773         help="synth = CPU-mock label-consistent trajectories (default, no GPU); "
774         "detector = REAL shuttle trajectories via the resolved DetectorRunner "
775         "(needs videos/ in the corpus; honours detector_impl).",
776     )
777     pt.add_argument(
778         "--segmenter",
779         default="wasb_hybrid",
780         help="trajectory hybrid whose detector is used for --trajectory-source
detector",
781     )
782     pt.add_argument(
783         "--floor", default=None, help="floor baseline JSON (local path or storage URI)"
784     )
785     pt.add_argument("--config", default=None)
786     pt.add_argument("--scratch", default=None)
787     pt.add_argument(
788         "--fps",
789         type=float,
790         default=30.0,
791         help="synth trajectory fps (detector probes the video)",
792     )
793     pt.add_argument(
794         "--frame-width",
795         type=float,
796         default=1280.0,
797         help="synth trajectory frame width (detector probes the video)",
798     )
799     pt.add_argument("--set", action="append", default=[], metavar="k=v")
800     pt.set_defaults(func=cmd_train)
801     return p
802
803
804     def _setup_logging(verbose: bool) -> None:
805         """Configure logging for the worker CLI so the pipeline's existing INFO/ERROR
diagnostics are
806         VISIBLE ? the worker previously ran with no logging config, so every
segmenter/detector log
807         (detector choice, 'Running native WASB inference', availability failures) was
silently dropped.
808         That is exactly why a 0-rally cold-start gave nothing to analyse. Configured here
(the CLI
809         entry), not at import, so importing the module never reconfigures a caller's
logging."""
810         # force=True: basicConfig is a no-op if the root already has handlers (e.g. a
library
811         # configured logging at import), which would leave the level at WARNING and re-
suppress the
812         # diagnostics. Forcing reconfiguration means the worker reliably controls its own
verbosity.

```

```

813         logging.basicConfig(
814             level=logging.DEBUG if verbose else logging.INFO,
815             format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
816             datefmt="%H:%M:%S",
817             force=True,
818         )
819
820
821     def main(argv=None) -> int:
822         args = build_parser().parse_args(argv)
823         _setup_logging(getattr(args, "verbose", False))
824         return args.func(args)
825
826
827     if __name__ == "__main__":
828         sys.exit(main())
829

```

6. assistant (2026-07-07T14:45:30.769Z)

Now let me look at `emit\_indexer\_report` and how `val\_context` is consumed, plus the orchestration ingest path.

7. user (2026-07-07T14:45:32.878Z)

No files found

8. user (2026-07-07T14:45:33.031Z)

```

__init__.py
__pycache__
harvest.py
run_signals.py
spec.yaml

```

9. user (2026-07-07T14:45:39.541Z)

No files found

10. user (2026-07-07T14:45:40.156Z)

No matches found

11. user (2026-07-07T14:45:52.029Z)

```

3be0935 docs(runlog): DEPLOY_REPORT addendum ? control-plane redeployed to the forwarding image (#510)
66756ad chore(serving): forward WASB decode/tuning knobs to worker + release hardening (#509)
143dbb9 feat(serving): default cloud-serving to nvdec_resident decode (#508)
44a2f47 feat(detectors): GPU-resident WASB NVDEC decode behind default-OFF decode_backend flag (#507)
17d844b docs(runlog): alpha-tester "All 1 v 1" cloud-serving run ? timeout diagnosed + downsample fix proven (#504)
---REPORTING INIT---
"""Per-video observability reports for the indexer.

`obsreport` is a SOFT dependency: if it is not installed, this module degrades to a
no-op and the pipeline runs exactly as before. The single entrypoint
`emit_indexer_report(...)` is designed to be called from a `finally` block and
NEVER raises ? emitting a report must not be able to break a processing run.
"""

```

```

from __future__ import annotations

```

```

from typing import Any, Dict, List, Optional

from . import run_signals # no obsreport dependency ? always importable

try: # soft dependency ? never hard-fail the pipeline on a reporting import
    import obsreport # noqa: F401

    from . import harvest

    OBSREPORT_AVAILABLE = True
except Exception: # pragma: no cover - exercised only when obsreport is absent
    OBSREPORT_AVAILABLE = False

def report_enabled(config: Optional[Dict[str, Any]]) -> bool:
    return bool((config or {}).get("report", {}).get("enabled", True))

def emit_indexer_report(
    *,
    db,
    video_id: str,
    video_path: Optional[str],
    config: Optional[Dict[str, Any]],
    val_results: List[Any],
    val_context: Optional[Dict[str, Any]] = None,
    segmenter_requested: Optional[str] = None,
    segmenter_actual: Optional[str] = None,
    was_reingest: bool = False,
    segmentation_ran: bool = True,
) -> Optional[Dict[str, str]]:
    """Harvest + write the report. Returns the written file paths, or None if
    reporting is unavailable/disabled/failed. Never raises."""
    config = config or {}
    # Accept either a plain dict or a typed config object (e.g. backend.config.AppConfig).
    # Normalize to a plain nested dict so harvest's dict access works uniformly.
    if hasattr(config, "model_dump"):
        try:
            config = config.model_dump(mode="json")
        except Exception:
            config = {}
    # Consume run signals unconditionally (even when reporting is off) so the
    # process-local store never accumulates entries for finished runs.
    signals = run_signals.pop(video_id)
    if not OBSREPORT_AVAILABLE or not report_enabled(config):
        return None
    try:
        play_segments = db.query_play_segments(video_id, {}) if db is not None else []
        candidates = db.query_candidate_segments(video_id) if db is not None else []
        rec = harvest.record_run(

```

12. assistant (2026-07-07T14:46:04.610Z)

Let me look at the orchestration ingest path and the validation gate.

13. user (2026-07-07T14:46:05.117Z)

```

1      """Processing / compile orchestration, extracted from ``backend.main`` (P23; re-cut of
#390).
2
3      The full hash -> validate -> segment -> report pipeline (``_execute_processing``), the
4      Cloud Run dispatch + ingest path (``_maybe_dispatch_remote`` /
``_ingest_remote_segments``),
5      and the shared compile resolution + worker handler (``_resolve_compile`` /
``_execute_compile``).

```

```

6      Extracted verbatim from ``backend.main`` to shrink the orchestrator; behavior is
unchanged.
7
8      SEAM DISCIPLINE (same convention as ``backend.api.job_worker``): anything that must
remain
9      monkeypatchable-on-``backend.main`` or that lives in ``backend.main`` (the transitional
10     ``db_instance``/``global_config`` globals and the ``ProcessRequest`` model) is resolved
through
11     the ``backend.main`` module object at call time (``import backend.main as _main``),
never as a
12     bare local. ``backend.main`` re-exports every name below, so
``monkeypatch.setattr(backend.main,
13     ...)`` and ``import backend.main as m; m._execute_processing(...)`` keep working
unchanged.
14     This module must NOT import ``backend.main`` at module level (main imports us for the
re-export).
15     Routed seams (rebind-patched on ``backend.main`` by tests): ``global_config``,
``db_instance``,
16     ``ProcessRequest``, ``compute_video_id``, ``emit_indexer_report``, ``VideoCompiler``.
17     Object-attribute patches (``m.storage.fetch``, ``m.segmenter_registry.get``, ...) hit
the shared
18     objects and need no routing.
19     """
20
21     import csv
22     import json
23     import logging
24     import math
25     import os
26     import re
27     import tempfile
28     import threading
29     from typing import TYPE_CHECKING, Any, Dict, List, Optional, Tuple
30     from urllib.parse import quote
31
32     from backend import storage
33     from backend.config import AppConfig, StitchingConfig
34     from backend.pipeline.segmenters.base import segmenter_registry
35     from backend.pipeline.stitcher.watermark_i18n import localize_watermark
36     from backend.pipeline.validators.base import registry
37     from backend.results import Failure
38     from backend.utils.paths import output_base, safe_output_filename
39
40     if (
41         TYPE_CHECKING
42     ): # runtime import would be circular (main imports us for the re-export)
43         from backend.main import ProcessRequest # noqa: F401
44
45     logger = logging.getLogger(__name__)
46
47     # #466: serialize CPU/GPU-heavy LOCAL work ? in-process segmentation and the ffmpeg
compile stitch ?
48     # so that raising the drain's worker count (to overlap I/O-bound REMOTE Cloud Run
dispatch/poll) never
49     # accidentally parallelizes local work. A single self-hosted box serializes GPU work and
Gemini is
50     # rate-fragile; a remote dispatch is a bucket-poll and never holds this lane, so N
remote jobs overlap
51     # while local/compile jobs stay 1-at-a-time. It's a plain mutex (no nesting) ? no
deadlock; when the
52     # drain is serial (max_workers=1) it's an uncontended no-op = today's behaviour.
53     _LOCAL_COMPUTE_LANE = threading.BoundedSemaphore(1)
54
55
56     def _finalize_reingest(db, video_id: str, segments, play_wm: int, cand_wm: int) -> None:

```

```

57         """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
58
59         On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
60         fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
61         and
62         keep the prior good results intact ? a failed/empty re-run never destroys prior
63         data.
64         """
65         if segments:
66             db.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
67         else:
68             db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
69
70     def _validation_failure_message(failures: List[Any]) -> str:
71         details = []
72         for failure in failures:
73             name = getattr(failure, "validator_name", None) or "validation"
74             message = (getattr(failure, "message", None) or "").strip()
75             details.append(f"{name}: {message}" if message else str(name))
76         if not details:
77             return "Video failed validation checks."
78         return "Video failed validation: " + "; ".join(details)
79
80     def _report_config_for_input(config: Any, raw_input: str) -> Any:
81         """Return report config that keeps remote-input reports out of disposable
82         scratch."""
83         storage_config = (
84             config.get("storage")
85             if isinstance(config, dict)
86             else getattr(config, "storage", None)
87         )
88         try:
89             if storage.input_is_local(raw_input, storage_config):
90                 return config
91         except ValueError:
92             return config
93         try:
94             cfg = (
95                 config.model_dump(mode="json")
96                 if hasattr(config, "model_dump")
97                 else dict(config or {})
98             )
99         except Exception:
100             cfg = {}
101         report = dict(cfg.get("report") or {})
102         if report.get("output_dir"):
103             return config
104         report["output_dir"] = os.path.join(output_base(config), "reports")
105         cfg["report"] = report
106         return cfg
107
108     def _ingest_remote_segments(
109         db, video_id: str, outputs_uri: str, storage_config: dict, *, max_rows: int = 20000
110     ) -> int:
111         """Read a cloud job's ``intervals.csv`` (start,end,shot_count,ending_reason) from
112         the bucket and
113         persist each rally via the SAME ``db.add_play_segment`` the in-process segmenters
114         call ? so
115         ``/api/query`` + ``/api/compile`` read the rows with zero knowledge of where they
116         were produced.
117         Returns the count ingested. Mirrors ``cloud_run._read_json``'s fetch-to-a-real-dst
118         pattern (cloud

```

```

115         backends REQUIRE a dst). Hardened against a hostile/corrupt worker output: a
malformed row is
116         skipped, non-finite / overflowing values are rejected, the loop is bounded, and the
temp dir is
117         cleaned. May raise (missing/unreadable CSV, fetch error) ? the caller treats that as
a job failure
118         and prunes any partial rows (destroy-AFTER-confirm)."""
119         uri = f"{outputs_uri.rstrip('/')}/intervals.csv"
120         ref = storage.parse_uri(uri)
121         n = 0
122         with tempfile.TemporaryDirectory(prefix="cr-ingest-") as td:
123             local = storage.fetch(
124                 uri, os.path.join(td, ref.name or "intervals.csv"), config=storage_config
125             )
126             with open(local, newline="", encoding="utf-8") as f:
127                 for row in csv.DictReader(f):
128                     if n >= max_rows:
129                         logger.warning(
130                             "[API] cloud ingest hit the %d-row cap (%s) ? ignoring the
rest.",
131                             max_rows,
132                             uri,
133                         )
134                         break
135                     try:
136                         start, end = float(row["start"]), float(row["end"])
137                     except (KeyError, TypeError, ValueError):
138                         continue
139                     if not (math.isfinite(start) and math.isfinite(end)):
140                         continue # reject inf/nan times rather than persist a junk rally
141                     meta: Dict[str, Any] = {
142                         "source": "cloud_run",
143                         "ending_reason": (row.get("ending_reason") or "").strip(),
144                     }
145                     sc = (row.get("shot_count") or "").strip()
146                     if sc:
147                         try:
148                             meta["shot_count"] = int(
149                                 float(sc)
150                             ) # OverflowError on 'inf'/'1e400'
151                         except (ValueError, OverflowError):
152                             meta["shot_count"] = sc
153                     db.add_play_segment(video_id, start, end, metadata=meta)
154                     n += 1
155         return n
156
157
158     def _cloud_available(cfg) -> bool:
159         """Whether THIS server can satisfy a per-request ``compute="cloud"`` dispatch (item
3).
160
161         True when the control plane is wired for Cloud Run: either the configured default
target is
162         already ``cloud_run`` (the operator chose cloud serving), or the Cloud Run env
coordinates
163         (``CLOUD_RUN_JOB`` + ``GOOGLE_CLOUD_PROJECT``) and an output bucket
(``storage.output_root``)
164         are present so a forced per-request override can actually dispatch + collect a
result.
165
166         Surfaced in ``/api/capabilities`` so the SPA only offers the cloud toggle when it
would work."""
167         deployment = getattr(cfg, "deployment", None)
168         if (getattr(deployment, "compute_target", "") or "") == "cloud_run":
169             return True

```



```

170     has_job = bool(
171         os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT")
172     )
173     out_root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
174     return bool(has_job and out_root)
175
176
177     def _wait_progress_message(seg_name: str, elapsed_sec: float) -> str:
178         """The live job.progress line while a cloud GPU run is in flight (#482). Honest and
179         minute-granular: the control plane knows ELAPSED time, not frames ? frame-level %
180         comes
181         with the worker-side heartbeat (the #482 follow-up). Under a minute reads as
182         "starting"
183         (queue/cold-start), so the very first tick already shows movement."""
184         minutes = int(elapsed_sec) // 60
185         if minutes < 1:
186             return f"segmenting ({seg_name}) on cloud GPU ? starting up"
187             return f"segmenting ({seg_name}) on cloud GPU ? {minutes} min elapsed"
188
189     def submit_time_video_id(input_ref, cfg) -> Optional[str]:
190         """The video's md5 (the canonical ``video_id``) resolvable CHEAPLY at submit time ?
191         gs://
192         object metadata only, one stat, no bytes (#484). ``None`` ? unknown: local files
193         keep
194         deferred hashing on the drain (DEFERRED_HARDENING #16), and composite objects carry
195         no
196         ``md5Hash``. Never raises ? a metadata miss must not block a submit."""
197         if getattr(input_ref, "backend", "") != "gcs":
198             return None
199         sc = _storage_settings(cfg)
200         if not sc:
201             return None
202         try:
203             st = storage.get_storage("gcs", config=sc).stat(input_ref)
204             return getattr(st, "md5", None) or None
205         except Exception: # noqa: BLE001 ? best-effort identity, the drain still hashes
206             logger.info(
207                 "[API] submit-time md5 unavailable for %s (metadata stat failed)",
208                 getattr(input_ref, "uri", input_ref),
209                 exc_info=True,
210             )
211             return None
212
213     def _cached_process_payload(
214         video_id: str, validation_results, segments, requested_compute
215     ) -> Dict[str, Any]:
216         """The /api/process result for a video whose detection already exists ? ONE shape
217         for
218         both producers (the drain's DB-cache short-circuit and the submit-time cache,
219         #484)."""
220         return {
221             "video_id": video_id,
222             "status": "success",
223             "message": (
224                 f"Reused {len(segments)} cached play segments. "
225                 "Pass force=true to reprocess this video."
226             ),
227             "results": validation_results or [],
228             "play_segments": segments,
229             "cached": True,
230             "compute": {"path": "cached", "requested": requested_compute},
231         }

```

```

228
229 def _read_bucket_json(uri: str, storage_config: dict) -> Optional[Dict[str, Any]]:
230     """Fetch + parse one small JSON object (cloud backends require a real dst ? the
231     ``_ingest_remote_segments`` pattern). Raises on fetch/parse errors; callers
decide."""
232     ref = storage.parse_uri(uri)
233     with tempfile.TemporaryDirectory(prefix="cr-report-") as td:
234         local = storage.fetch(
235             uri, os.path.join(td, ref.name or "report.json"), config=storage_config
236         )
237         with open(local, encoding="utf-8") as f:
238             loaded = json.load(f)
239         return loaded if isinstance(loaded, dict) else None
240
241
242 # One re-hydrate at a time per video_id (#492 review blocker 1): two concurrent cache
hits
243 # from an empty DB must not both ingest intervals.csv ? the loser waits, then finds the
244 # winner's rows in the DB and reuses them. Process-wide is enough: the CP is single-
instance
245 # (maxScale=1, spec D8) and SQLite serializes cross-process writers anyway.
246 _REHYDRATE_LOCKS: Dict[str, threading.Lock] = {}
247 _REHYDRATE_LOCKS_GUARD = threading.Lock()
248
249
250 def _rehydrate_lock(video_id: str) -> threading.Lock:
251     with _REHYDRATE_LOCKS_GUARD:
252         return _REHYDRATE_LOCKS.setdefault(video_id, threading.Lock())
253
254
255 def probe_process_cache(db, cfg, video_id: str) -> bool:
256     """READ-ONLY: does completed detection for ``video_id`` already exist (DB or
bucket)?
257
258     Safe to call before the atomic submit claim ? it writes nothing (#492 review blocker
1
259     ordering). Used to decide whether a submit can skip the local-scratch fetch guard
260     (blocker 2): a True probe means the request should never need CP scratch. Best-
effort:
261     errors report False (? the normal dispatch path with its guards)."""
262     try:
263         existing = db.get_video(video_id)
264         if (
265             existing
266             and existing.get("status") == "processed"
267             and db.query_play_segments(video_id, {})
268         ):
269             return True
270         sc = _storage_settings(cfg)
271         if not sc or (sc.get("backend") or "local") == "local":
272             return False
273         output_root = (sc.get("output_root") or "").rstrip("/")
274         if not output_root:
275             return False
276         report_uri = f"{output_root}/outputs/{video_id}/report.json"
277         if not storage.exists(report_uri, config=sc):
278             return False
279         report = _read_bucket_json(report_uri, sc)
280         return bool(report and report.get("status") == "success")
281     except Exception: # noqa: BLE001 ? a failed probe degrades to the guarded dispatch
path
282         logger.warning(
283             "[API] cache probe failed for %s ? treating as a miss", video_id,
exc_info=True
284         )

```

```

285         return False
286
287
288     def cached_process_result(
289         db, cfg, video_id: str, video_path: str, requested_compute=None
290     ) -> Optional[Dict[str, Any]]:
291         """A ready-to-serve /api/process result when this video's detection ALREADY exists ?
292         the DB first, then the bucket (`outputs/<md5>/report.json` with `status=success`
293         re-ingests via `_ingest_remote_segments`): **the DB is a cache, the bucket is the
294         source of truth** (#484 instant re-runs; the #485 re-hydrate mechanism). `None` ?
295         no
296         completed work found (or the check failed) ? the caller dispatches normally. The
297         bucket
298         branch mirrors the real cloud ingest exactly (watermarks + `_finalize_reingest`),
299         so a
300         partial earlier ingest is replaced, never double-counted.
301
302         Concurrency (#492 review blocker 1): the whole check-then-ingest runs under a
303         per-`video_id` lock ? a concurrent caller blocks, then finds the winner's rows via
304         the
305         in-lock DB re-check and reuses them (one segment set, ever). Callers must hold a
306         CLAIMED
307         job before invoking (writes happen post-claim)."""
308         try:
309             with _rehydrate_lock(video_id):
310                 existing = db.get_video(video_id)
311                 if existing and existing.get("status") == "processed":
312                     segs = db.query_play_segments(video_id, {})
313                     if segs:
314                         return _cached_process_payload(
315                             video_id,
316                             existing.get("validation_results", []),
317                             segs,
318                             requested_compute,
319                         )
320                 sc = _storage_settings(cfg)
321                 if not sc or (sc.get("backend") or "local") == "local":
322                     return None
323                 output_root = (sc.get("output_root") or "").rstrip("/")
324                 if not output_root:
325                     return None
326                 outputs_uri = f"{output_root}/outputs/{video_id}"
327                 report_uri = f"{outputs_uri}/report.json"
328                 if not storage.exists(report_uri, config=sc):
329                     return None
330                 report = _read_bucket_json(report_uri, sc)
331                 if not report or report.get("status") != "success":
332                     return None
333                 if not video_path:
334                     # Read-path re-hydrate (#485): a wiped DB has no source record. The
335                     report
336                     # carries the worker's --video-uri (P3); older reports fall back to the
337                     # staged-canonical videos/<md5>/ prefix; else "" ? the segments still
338                     # recover, and the next same-md5 submit re-binds the truthful path via
339                     # add_video's upsert.
340                     video_path = str(report.get("video_uri") or "") or _first_object_under(
341                         f"{output_root}/videos/{video_id}/", sc
342                     )
343                 play_wm, cand_wm = db.segment_watermarks(video_id)
344                 db.add_video(video_id, video_path, "processed", [])
345                 n = _ingest_remote_segments(db, video_id, outputs_uri, sc)
346                 if n <= 0:
347                     return None
348                 segments = [
349                     s

```

```

344         for s in db.query_play_segments(video_id, {}):
345             if int(s.get("id", 0)) > play_wm:
346                 ]
347             _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
348             if not segments:
349                 return None
350             logger.info(
351                 "[API] re-hydrated %d segments for video_id=%s from %s (submit-time
cache hit)",
352                 n,
353                 video_id,
354                 outputs_uri,
355             )
356             return _cached_process_payload(video_id, [], segments, requested_compute)
357     except Exception: # noqa: BLE001 ? cache misses must degrade to a normal dispatch
358         logger.warning(
359             "[API] submit-time cache check failed for %s ? dispatching normally",
360             video_id,
361             exc_info=True,
362         )
363     return None
364
365
366     def _first_object_under(prefix_uri: str, storage_config: dict) -> str:
367         """The first object URI under a bucket prefix, or "" ? best-effort source
recovery."""
368         try:
369             return next(iter(storage.list_uris(prefix_uri, config=storage_config)), "")
370         except Exception: # noqa: BLE001 ? recovery nicety, never a failure
371             return ""
372
373
374     _MD5_HEX_RE = re.compile(r"[0-9a-f]{32}")
375
376
377     def read_path_rehydrate(db, cfg, video_id: str) -> bool:
378         """#485, the read half of "DB = cache, bucket = truth": before a rally READ reports
379         nothing for an md5-shaped id, try re-hydrating the video from ``outputs/<md5>/`` ?
380         so
381         ``?video=<md5>`` lands on grounded state after any restart/redeploy wipe. True ? the
DB
382         has the video's rows now (re-query). Non-md5 ids return False without touching the
383         bucket; every failure degrades to False (the read just reports what the DB has)."""
384         vid = (video_id or "").strip().lower()
385         if not _MD5_HEX_RE.fullmatch(vid):
386             return False
387         if not probe_process_cache(db, cfg, vid):
388             return False
389         return cached_process_result(db, cfg, vid, "") is not None
390
391     def _maybe_dispatch_remote(
392         cfg,
393         db,
394         req: "ProcessRequest",
395         video_id: str,
396         seg_name: str,
397         val_results,
398         play_wm: int,
399         cand_wm: int,
400         notify,
401     ) -> Optional[Tuple[Dict[str, Any], bool]]:
402         """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
``deployment.compute_target
403         == "cloud_run"`` , run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally

```

```

intervals into
404     THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.
405
406     Returns `` (response_dict, ran) `` when the cloud path is taken ? ``ran`` is whether
the cloud job
407     actually EXECUTED (``False`` for a pre-dispatch config rejection, so the
observability report
408     doesn't claim a fake segmentation run) ? or ``None`` to fall through to the in-
process segmenter
409     (**DEFAULT-OFF**: ``get_compute_backend`` returns ``None`` for every
non-``cloud_run`` target, so
410     the in-process path stays byte-identical)."""
411     from backend.compute import ComputeJobSpec, get_compute_backend
412
413     def _failed(msg: str, ran: bool) -> Tuple[Dict[str, Any], bool]:
414         # Shape-match the in-process segmenter-fail branch + drop any partial NEW rows
(id > play_wm);
415         # prior good rows (id <= play_wm) are preserved (destroy-AFTER-confirm).
416         db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
417         # Provenance even on failure, and HONEST about what ran: `path` is "cloud_run"
only when the
418         # job actually dispatched + executed remotely (ran=True, e.g. a non-zero worker
rc / ingest
419         # error); a pre-dispatch reject (ran=False) ran NOWHERE ? "not_run". `target`
records the
420         # intended backend, `requested` the picker choice, `dispatched` whether the
remote job ran.
421         return (
422             {
423                 "video_id": video_id,
424                 "status": "failed",
425                 "message": msg,
426                 "results": [r.model_dump() for r in val_results],
427                 "play_segments": [],
428                 "compute": {
429                     "path": "cloud_run" if ran else "not_run",
430                     "requested": req.compute,
431                     "target": "cloud_run",
432                     "dispatched": ran,
433                 },
434             },
435             ran,
436         )
437
438     # SPA compute-picker (item 3): a per-request choice overrides the server's
configured default.
439     # "local" forces the in-process segmenter; "cloud" forces a Cloud Run dispatch
(guarded by
440     # _cloud_available so we fail clearly rather than silently running local); None /
unknown ? default.
441     choice = (req.compute or "").strip().lower()
442     if choice and choice not in ("local", "cloud"):
443         logger.warning(
444             "[API] ignoring unknown compute=%r (expected 'local'/'cloud'); using server
default.",
445             req.compute,
446         )
447     choice = ""
448     if choice == "local":
449         return None # explicit "run on my machine" ? in-process regardless of the
server's target
450     if choice == "cloud":
451         if not _cloud_available(cfg):
452             return _failed(
453                 "cloud-serving was requested but this server isn't configured for it "

```

```

454         "(no Cloud Run target). Pick 'run on my machine', or configure cloud
serving.",
455         False,
456     )
457     compute = get_compute_backend(cfg, target_override="cloud_run")
458 else:
459     compute = get_compute_backend(
460         cfg
461     ) # server default (default-OFF unless cloud_run)
462 if compute is None:
463     return None # default-OFF ? run the in-process segmenter below (unchanged)
464
465 storage_cfg = getattr(cfg, "storage", None)
466
467 # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local
to this box.
468 try:
469     input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
470 except ValueError as e:
471     return _failed(
472         f"cloud-serving: unresolvable input '{req.video_path}': {e}", False
473     )
474 if input_ref.backend == "local":
475     # A browser upload lands on THIS box's local disk; the Cloud Run job (a
different machine)
476     # can't read it. Stage it UP to the input bucket first, then dispatch from there
(#461). Only
477     # possible when a CLOUD input_root is configured; without one we genuinely can't
stage ? fail.
478     # Classify the ROOT ITSELF (parse_uri), not storage.input_backend: the staged
URI below is
479     # built by joining under input_root, and storage.put routes by
parse_uri(staged_uri).backend ?
480     # so an explicit cloud root (gs://?, s3://?) stages fine with input_backend left
at its
481     # default, while a bare/local-looking root can't stage no matter what
input_backend says
482     # (resolve_input_uri only consults input_backend when input_root is EMPTY, and
an empty root
483     # gives us nothing to build the staged URI under).
484     input_root = (getattr(storage_cfg, "input_root", "") or "").strip()
485     try:
486         root_backend = (
487             storage.parse_uri(input_root).backend if input_root else "local"
488         )
489     except ValueError: # unsupported scheme in input_root ? nowhere real to stage
to
490         root_backend = "local"
491     if root_backend == "local":
492         return _failed(
493             "cloud-serving needs a cloud input (e.g. gs://?), or a cloud
storage.input_root to "
494             "stage local uploads into; a path local to this machine can't be read by
the Cloud "
495             "Run job.",
496             False,
497         )
498     local_src = input_ref.key # resolved local path of the upload
499     staged_uri =
f"{input_root.rstrip('/')}/{video_id}/{os.path.basename(local_src)}"
500     try:
501         notify("staging upload to bucket")
502         # Thread the active storage settings so backend-specific config survives (S3
endpoint_url/
503         # region/addressing for R2·D0·MinIO; GCS project/client) ? not a fresh

```

```

default client.
504         storage.put(
505             local_src,
506             staged_uri,
507             config=(
508                 storage_cfg.model_dump() if storage_cfg is not None else None
509             ),
510         )
511     except Exception as e: # noqa: BLE001 ? a staging failure is a clean job
failure, not a 500
512         logger.error(
513             "[API] cloud-serving: failed to stage %s -> %s: %s",
514             local_src,
515             staged_uri,
516             e,
517             exc_info=True,
518         )
519         return _failed(
520             f"cloud-serving: failed to stage the uploaded file to {input_root}:
{e}",
521             False,
522         )
523     # Repoint BOTH the dispatch and the DB video record at the staged bucket object:
the L4 job
524     # DETECTs from it, and a later /api/compile re-fetches the same source (the
local upload is
525     # gone once this instance recycles). add_video is an UPSERT ? it only updates
filepath.
526     req.video_path = staged_uri
527     input_ref = storage.resolve_input_ref(staged_uri, storage_cfg)
528     db.add_video(
529         video_id, staged_uri, "processed", [r.model_dump() for r in val_results]
530     )
531     out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
532     if not out_root:
533         return _failed(
534             "cloud-serving needs storage.output_root set to a bucket (e.g. gs://your-
bucket) "
535             "? that's where the Cloud Run job writes the result.",
536             False,
537         )
538
539     notify("dispatching (cloud_run)")
540     logger.info(
541         "[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
542         req.video_path,
543         out_root,
544     )
545     # Forward the engine's AI-handoff stance so the cloud job runs the SAME detection:
with
546     # skip_ai_handoff=True the WASB windows become rallies directly (no Gemini key
needed); without
547     # forwarding the cloud job would default to the handoff and yield 0 rallies when it
has no key.
548     # (player_pool / max_frames are still dropped on the cloud path ? a separate
documented follow-up.)
549     overrides = []
550     idx_cfg = getattr(cfg, "indexing", None)
551     sk = getattr(idx_cfg, "skip_ai_handoff", None)
552     if sk is not None:
553         overrides.append(f"indexing.skip_ai_handoff={'true' if sk else 'false'}")
554     # Forward the resolved WASB serving knobs so the PRESET governs the worker
(#506/#507). The
555     # worker Job has no DEPLOYMENT_PROFILE, so it does NOT apply the cloud-serving
preset itself ?

```

```

556         # without forwarding, its decode_backend + GPU-feed tuning silently fall back to the
model
557         # defaults (cpu, batch 8) regardless of what the control-plane resolved.
decode_backend is
558         # cuda-only (validator-enforced) and requires a worker image that understands
--decode-backend
559         # (? #507); batch_size/prefetch_batches are forwarded only when set (None = the
parity default).
560         wasb_cfg = getattr(idx_cfg, "wasb", None)
561         if wasb_cfg is not None:
562             decode_backend = getattr(wasb_cfg, "decode_backend", None)
563             if decode_backend:
564                 overrides.append(f"indexing.wasb.decode_backend={decode_backend}")
565             batch_size = getattr(wasb_cfg, "batch_size", None)
566             if batch_size:
567                 overrides.append(f"indexing.wasb.batch_size={batch_size}")
568             prefetch = getattr(wasb_cfg, "prefetch_batches", None)
569             if prefetch:
570                 overrides.append(f"indexing.wasb.prefetch_batches={prefetch}")
571         # Tell the worker to SKIP its redundant re-validation: THIS control-plane is the
authoritative
572         # gate ? it already ran the validators with its resolved config just above (only a
PASS reaches
573         # here; a failure returned "failed" before dispatch). A worker re-run would decode
the video a
574         # second time and, because its config isn't fully forwarded, can resolve DIFFERENT
thresholds and
575         # REJECT a clip this gate already passed (observed 2026-07-07: passed at
min_blur_threshold=8.0,
576         # then worker-rejected at its default 20.0 after a successful GPU dispatch).
577         spec = ComputeJobSpec(
578             video_uri=req.video_path,
579             out_uri=out_root,
580             segmenter=seg_name,
581             config_overrides=tuple(overrides),
582             skip_validation=True,
583         )
584         try:
585             # Dispatch the Cloud Run Job + bucket-poll to completion. on_wait is the live
586             # heartbeat (#482): job.progress moves on every poll tick instead of freezing at
587             # "segmenting" for the whole GPU run (the 2026-07-05 silent 22-min spinner) ?
which
588             # also gives the SPA a stall signal to time out on, instead of a blind wall
clock.
589             result = compute.run_infer(
590                 spec,
591                 on_wait=lambda elapsed: notify(_wait_progress_message(seg_name, elapsed)),
592             )
593         except Exception as e: # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
594             logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
595             return _failed(f"cloud dispatch error: {e}", True)
596         if result.returncode != 0:
597             return _failed(f"cloud job failed (returncode {result.returncode}).", True)
598         if result.video_id and result.video_id != video_id:
599             # The worker re-hashes the bucket bytes; a divergence means the input changed
between this
600             # box's hash and the worker's ? ingest under the LOCAL id but flag it so it's
diagnosable.
601             logger.warning(
602                 "[API] cloud video_id %s != local %s ? input bytes differ between the API
hash "
603                 "and the worker hash; ingesting under the local id.",
604                 result.video_id,
605                 video_id,

```



```

606         )
607
608     notify("ingesting (cloud_run)")
609     storage_dict = (
610         storage_cfg.model_dump() # type: ignore[union-attr] # hasattr-guarded; mypy
can't narrow
611         if hasattr(storage_cfg, "model_dump")
612         else {}
613     )
614     try:
615         _ingest_remote_segments(db, video_id, result.outputs_uri, storage_dict)
616     except Exception as e: # noqa: BLE001 ? a missing/partial/corrupt result must not
500 or leak rows
617         logger.error("[API] cloud ingest failed: %s", e, exc_info=True)
618         return _failed(
619             f"cloud-serving: could not read the result from {result.outputs_uri}: {e}",
620             True,
621         )
622
623     # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then
drops the old ones.
624     segments = [
625         s for s in db.query_play_segments(video_id, {}) if int(s.get("id", 0)) > play_wm
626     ]
627     _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
628     # Compute-path PROVENANCE: the GCP-verifiable proof the cloud path ran. `execution`
is the real
629     # Cloud Run execution name (cross-check: gcloud run jobs executions describe
<execution>).
630     provenance = {
631         "path": "cloud_run",
632         "requested": req.compute,
633         "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
634         "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
635         "execution": result.execution,
636         "outputs_uri": result.outputs_uri,
637     }
638     logger.info(
639         "[COMPUTE] cloud_run dispatched: execution=%s job=%s region=%s out=%s
video_id=%s",
640         result.execution,
641         provenance["job"],
642         provenance["region"],
643         result.outputs_uri,
644         video_id,
645     )
646     return (
647         {
648             "video_id": video_id,
649             "status": "success",
650             "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}').",
651             "results": [r.model_dump() for r in val_results],
652             "play_segments": segments,
653             "compute": provenance,
654         },
655         True,
656     )
657
658
659     def _execute_processing(
660         config=None, db=None, req=None, progress=None
661     ) -> Dict[str, Any]:
662         """The full hash ? validate ? segment ? report pipeline for one video.
663

```

```

664     Accepts explicit ``config``/``db`` (the new app-factory pattern) OR reads the
665     module-level ``global_config``/``db_instance`` (backward-compatible with existing
666     tests that pass only ``req``). Route handlers always pass explicit params; older
667     test code that calls ``_execute_processing(req)`` still works.
668
669     ``progress`` is an optional callable(stage: str) used to surface coarse job
progress.
670     Raises on unexpected internal errors ? the caller records those as a job failure
671     ""
672     import backend.main as _main # call-time seam: main's globals/model stay patchable
673
674     # Backward-compatible shim: if called as _execute_processing(req) (old pattern),
675     # shift positional args ? req arrives in position 0 (config), progress in position 1
(db).
676     if config is not None and not isinstance(config, AppConfig):
677         # Old signature: _execute_processing(req, progress=None)
678         if req is None and db is not None and isinstance(db, _main.ProcessRequest):
679             req = db
680             db = None
681         elif req is None and isinstance(config, _main.ProcessRequest):
682             req = config
683             config = None
684     # Resolve config/db from module globals when not passed explicitly
685     if config is None:
686         config = _main.global_config
687     if db is None:
688         db = _main.db_instance
689
690     notify = progress or (lambda stage: None)
691
692     notify("hashing")
693     # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
694     # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
695     # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
696     # consumers (hash / validators / segmenter) use the resolved local path.
697     local_video = ""
698     try:
699         local_video = storage.acquire_local_input(
700             req.video_path, getattr(config, "storage", None)
701         )
702         video_id = _main.compute_video_id(local_video)
703     except Exception:
704         storage.cleanup_acquired_local_input(
705             req.video_path, local_video, getattr(config, "storage", None)
706         )
707         raise
708     try:
709         existing_video = db.get_video(video_id)
710     except Exception:
711         storage.cleanup_acquired_local_input(
712             req.video_path, local_video, getattr(config, "storage", None)
713         )
714         raise
715     was_reingest = existing_video is not None
716
717     # typed AppConfig: attribute access for the default segmenter (with safe fallback)
718     default_seg = getattr(
719         getattr(config, "indexing", None), "default_segmenter", "motion"
720     )
721     seg_name = req.segmenter or default_seg
722     segmenter_actual = None
723     segmentation_ran = False

```

```

724     val_results: List[Any] = []
725     val_context: Dict[str, Any] = {}
726     # Compute-path PROVENANCE (the validation method): which backend actually ran the
DETECT ?
727     # "in_process" once the local segmenter is reached, set on the response as
"cloud_run" by
728     # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
fail).
729     compute_actual: Optional[str] = None
730     response: Optional[Dict[str, Any]] = None
731     try:
732         if (
733             existing_video
734             and existing_video.get("status") == "processed"
735             and not req.force
736         ):
737             cached_segments = db.query_play_segments(video_id, {})
738             if cached_segments:
739                 logger.info(
740                     "[API] Reusing cached segmentation for video_id=%s; force=true
bypasses cache.",
741                     video_id,
742                 )
743             response = _cached_process_payload(
744                 video_id,
745                 existing_video.get("validation_results", []),
746                 cached_segments,
747                 getattr(req, "compute", None),
748             )
749             return response
750
751     # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for
the report)
752     notify("validating")
753     logger.info(f"Running validations for {req.video_path}...")
754     val_results, val_context = registry.run_all_with_context(local_video, config)
755     failures = [r for r in val_results if not r.passed]
756
757     if failures:
758         # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
759         # status; it no longer destroys the video's prior good segments.
760         db.add_video(
761             video_id,
762             req.video_path,
763             "failed",
764             [r.model_dump() for r in val_results],
765         )
766         response = {
767             "video_id": video_id,
768             "status": "failed",
769             "message": _validation_failure_message(failures),
770             "results": [r.model_dump() for r in val_results],
771         }
772         return response
773
774     # 2. Run segmenter & indexer
775     logger.info("[API] Video passed checks. Segmenting and indexing...")
776     db.add_video(
777         video_id, req.video_path, "processed", [r.model_dump() for r in val_results]
778     )
779
780     segmenter_cls = segmenter_registry.get(seg_name)
781     if not segmenter_cls:
782         # Submit-time validation already 400s unknown names; this is the defensive
783         # in-worker path (e.g. config default pointing at an unregistered

```

```

segmenter).
784         available = list(segmenter_registry.list_available().keys())
785         response = {
786             "video_id": video_id,
787             "status": "failed",
788             "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
789             "results": [r.model_dump() for r in val_results],
790             "play_segments": [],
791         }
792         return response
793
794         logger.info(f"[API] Using segmenter: {seg_name}")
795         notify(f"segmenting ({seg_name})")
796         segmenter_actual = seg_name
797         # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
798         # if the run yields segments; otherwise drop the new partial rows and keep the
old.
799         play_wm, cand_wm = db.segment_watermarks(video_id)
800         # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run,
run the heavy
801         # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
802         # DEFAULT-OFF: returns None for every non-cloud target ? the in-process
segmenter below runs
803         # byte-identically. (player_pool/max_frames aren't threaded to the worker
`infer` CLI yet, so
804         # they're dropped on the cloud path in this slice ? a documented follow-up.)
805         remote = _maybe_dispatch_remote(
806             config, db, req, video_id, seg_name, val_results, play_wm, cand_wm, notify
807         )
808         if remote is not None:
809             response, segmentation_ran = (
810                 remote # `ran` = did the cloud job actually execute (report)
811             )
812             return response # the cloud branch already stamped response["compute"]
(path=cloud_run)
813         # In-process from here on ? record the actual path for the provenance stamp in
`finally`.
814         compute_actual = "in_process"
815         segmenter = segmenter_cls(db, config)
816         try:
817             # Serialize the CPU/GPU-heavy in-process detection (see
_LOCAL_COMPUTE_LANE). A remote
818             # dispatch never reaches here (it returned above), so remote polls still
overlap freely.
819             with _LOCAL_COMPUTE_LANE:
820                 result = segmenter.process_video(
821                     video_id, local_video, req.player_pool, req.max_frames
822                 )
823         except Exception:
824             # A raising segmenter must not leave half-written rows: restore the pre-run
825             # state (same destroy-after-confirm contract as the Failure branch), then
let
826             # the job worker record the failure.
827             db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
828             raise
829         segmentation_ran = True
830
831         if isinstance(result, Failure):
832             logger.error(f"[API] Segmenter failed: {result.message}")
833             db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
834             response = {
835                 "video_id": video_id,

```

```

836         "status": "failed",
837         "message": f"Segmenter '{seg_name}' failed: {result.message}",
838         "results": [r.model_dump() for r in val_results],
839         "play_segments": [],
840     }
841     return response
842
843     segments = result.value if hasattr(result, "value") else result
844     notify("finalizing")
845     _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
846
847     response = {
848         "video_id": video_id,
849         "status": "success",
850         "message": f"Successfully indexed {len(segments)} play segments using
851 '{seg_name}' segmenter.", # type: ignore[arg-type] # hasattr-unwrap above; Success arm has
852         "results": [r.model_dump() for r in val_results],
853         "play_segments": segments,
854     }
855     return response
856 finally:
857     # Compute-path PROVENANCE stamp (the validation method): the cloud branch
858     already set
859     # response["compute"] (path=cloud_run + the Cloud Run execution); for every
860     other path stamp
861     # the actual path here so the job result PROVES which backend ran (in_process /
862     not_run) and
863     # what the picker REQUESTED ? a silent fallback (requested cloud, ran local) is
864     then visible.
865     if isinstance(response, dict):
866         response.setdefault(
867             "compute",
868             {
869                 "path": compute_actual or "not_run",
870                 "requested": getattr(req, "compute", None),
871             },
872         )
873     prov = response.get("compute", {}) if isinstance(response, dict) else {}
874     logger.info(
875         "[COMPUTE] video_id=%s requested=%s actual=%s dispatched=%s status=%s",
876         video_id,
877         getattr(req, "compute", None),
878         prov.get("path"),
879         prov.get("dispatched"),
880         (response or {}).get("status") if isinstance(response, dict) else None,
881     )
882     # Always emit the per-video observability report (next to the video, or to
883     # report.output_dir). Never raises. Surface the paths in the response. (The
884     compute-path
885     # proof lives on the job result + the [COMPUTE] log, not the report ?
886     obsreport's record_run
887     # doesn't surface arbitrary run signals.)
888     paths = _main.emit_indexer_report(
889         db=db,
890         video_id=video_id,
891         # Resolved LOCAL path, not the raw URI: obsreport writes the report next to
892         the
893         # video via os.path.abspath(video_path), and a remote URI (gs://, gcs://,
894         local://)
895         # collapses to a literal scheme dir at the repo root (see #397). The source
896         URI is
897         # still preserved in DB provenance via db.add_video.
898         video_path=local_video,
899         config=_report_config_for_input(config, req.video_path),

```

```

890         val_results=val_results,
891         val_context=val_context,
892         segmenter_requested=req.segmenter,
893         segmenter_actual=segmenter_actual,
894         was_reingest=was_reingest,
895         segmentation_ran=segmentation_ran,
896     )
897     if paths and isinstance(response, dict):
898         response["reports"] = paths
899     storage.cleanup_acquired_local_input(
900         req.video_path, local_video, getattr(config, "storage", None)
901     )
902
903
904     def _remote_input_size_bytes(input_ref, storage_config) -> Optional[int]:
905         """Best-effort size (bytes) of a remote storage source, or ``None`` if unknowable.
906
907         Used only to pre-reserve scratch for a fetch-to-local (P2). Never raises: a stat
908         that needs a
909         missing cloud SDK, the network, or a credential returns ``None`` so the free-space
910         guard degrades
911         to a no-op rather than blocking a job it cannot measure."""
912         try:
913             sc = (
914                 storage_config.model_dump()
915                 if hasattr(storage_config, "model_dump")
916                 else (storage_config or {})
917             )
918             backend = storage.get_storage(input_ref.backend, config=sc)
919             return int(backend.stat(input_ref).size)
920         except Exception as e: # noqa: BLE001 - measuring is best-effort; inability must
921             not block
922             logger.debug(
923                 "free-space: could not stat remote source %s: %s", input_ref.uri, e
924             )
925             return None
926
927
928     class _CompileError(Exception):
929         """A submit-time-validatable compile problem ? carries the HTTP status + detail so
930         the API path
931         maps it to an HTTPException (fast 4xx) and the worker path maps it to a failed-job
932         result."""
933
934         def __init__(self, status_code: int, detail: str):
935             super().__init__(detail)
936             self.status_code = status_code
937             self.detail = detail
938
939
940     def _resolve_compile(
941         db, cfg, video_id: str, segment_ids: List[int], output_filename: str
942     ):
943         """Shared compile resolution ? run at SUBMIT (fast-fail 4xx) AND in the WORKER (re-
944         resolve, since a
945         row could change between submit and run). Verifies the video exists, the requested
946         segments match
947         and survive the ``stitching.min_shot_count`` floor (segments WITHOUT a known
948         shot_count are exempt
949         so manual/legacy picks can't silently vanish), and the output name is traversal-
950         safe. Raises
951         ``_CompileError(404/400)``. Returns ``(video_row, kept_segments,
952         safe_out_name)``."""
953         video = db.get_video(video_id)
954         if not video:

```

```

945         raise _CompileError(404, "Indexed video record not found.")
946     # Reject path traversal / absolute names (os.path.join would otherwise write
anywhere on disk).
947     try:
948         out_name = safe_output_filename(output_filename)
949     except ValueError as e:
950         raise _CompileError(400, str(e)) from e
951
952     all_segments = db.query_play_segments(video_id, {})
953     matched = [s for s in all_segments if s["id"] in segment_ids]
954     if not matched:
955         raise _CompileError(400, "No matching play segments found to stitch.")
956
957     stitching = getattr(cfg, "stitching", None) or StitchingConfig()
958     kept = []
959     for s in matched:
960         meta = s.get("metadata") or {}
961         sc = meta.get("shot_count") if isinstance(meta, dict) else None
962         try:
963             if sc is not None and int(sc) < stitching.min_shot_count:
964                 continue
965             except (TypeError, ValueError):
966                 pass # unparseable count -> treat as unknown, keep
967             kept.append(s)
968     if len(kept) < len(matched):
969         logger.info(
970             "[compile] stitching.min_shot_count=%s: dropped %d/%d segments below the
floor.",
971             stitching.min_shot_count,
972             len(matched) - len(kept),
973             len(matched),
974         )
975     if not kept:
976         raise _CompileError(
977             400,
978             f"All {len(matched)} matching segments fall below "
979             f"stitching.min_shot_count={stitching.min_shot_count}.",
980         )
981     return video, kept, out_name
982
983
984 def reel_object_uri(cfg, video_id: str, name: str) -> str:
985     """The bucket URI a compiled reel is mirrored to for a direct signed-URL download
(#463), or "" when
986     the output isn't a cloud bucket (truly-local ? serve the local file). Shared by the
compile
987     mirror-upload and the /api/highlights redirect:
`<output_root>/highlights/<video_id>/<name>`. """
988     root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
989     if not root or "://" not in root:
990         return ""
991     return f"{root.rstrip('/')}/highlights/{video_id}/{name}"
992
993
994 def _storage_settings(cfg) -> Optional[Dict[str, Any]]:
995     """The active storage settings dict (per-backend config for the storage layer ? S3
endpoint/region,
996     GCS project/client/signed_url_ttl), or None. Threaded into storage calls so a non-
default backend
997     isn't silently constructed as a fresh default. """
998     sc = getattr(cfg, "storage", None)
999     return sc.model_dump() if sc is not None else None
1000
1001
1002 def signed_reel_url(cfg, video_id: str, name: str) -> Optional[str]:

```

```

1003         """A short-lived signed URL for a reel mirrored to the output bucket, so
/api/highlights can
1004         307-redirect off the app's response path (bypassing Cloud Run's ~32 MiB cap + the
ephemeral local
1005         /tmp; #463). None ? no bucket copy ? serve the local file. Uses the active storage
config for BOTH
1006         the existence check and the signing (extracted here to keep backend/main.py lean ?
P23/P24)."""
1007         reel_uri = reel_object_uri(cfg, video_id, name)
1008         if not reel_uri:
1009             return None
1010         settings = _storage_settings(cfg)
1011         if storage.exists(reel_uri, config=settings):
1012             return storage.signed_url(reel_uri, config=settings, method="GET")
1013         return None
1014
1015
1016     def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
1017         """Worker handler for a ``kind='compile'`` job (#329): re-resolve the segments,
stitch the reel,
1018         and return the {status, filepath, download_url} result the SPA polls for. Returns a
1019         ``status='failed'`` result for an EXPECTED problem (missing video / no segments / a
stitch error)
1020         so the SPA always gets a clean verdict instead of a worker-crash error."""
1021         import backend.main as _main # call-time seam: VideoCompiler stays patchable on
backend.main
1022
1023         notify = progress or (lambda stage: None)
1024         params = json.loads(job.get("params") or "{}")
1025         video_id = job.get("video_id") or ""
1026         segment_ids = params.get("segment_ids") or []
1027         locale = params.get("locale")
1028
1029         notify("resolving")
1030         try:
1031             video, kept, out_name = _resolve_compile(
1032                 db, cfg, video_id, segment_ids, params.get("output_filename", "")
1033             )
1034         except _CompileError as e:
1035             return {"status": "failed", "message": e.detail, "video_id": video_id}
1036
1037         stitching = getattr(cfg, "stitching", None) or StitchingConfig()
1038         output_dir = (
1039             getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
1040         )
1041         # Localize the branding watermark to the UI locale the SPA rendered in (recorded on
the job);
1042         # unknown/absent locale keeps the configured default unchanged.
1043         localized_watermark = localize_watermark(stitching.watermark, locale)
1044         compiler = _main.VideoCompiler(
1045             output_dir,
1046             begin_padding=stitching.begin_padding,
1047             end_padding=stitching.end_padding,
1048             watermark=localized_watermark,
1049         )
1050         time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
1051
1052         notify("stitching")
1053         local_src = ""
1054         try:
1055             # The stored `filepath` is the SOURCE ref (gs://, s3://, gdrive://, or a local
path). Resolve to
1056             # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-
scratch for a URI (the
1057             # same seam the process path uses); without it, compiling a bucket-ingested

```



```

video can't open gs://.
1058         local_src = storage.acquire_local_input(
1059             video["filepath"], getattr(cfg, "storage", None)
1060         )
1061         # The ffmpeg stitch is CPU-heavy ? serialize it so a raised drain worker count
(for remote
1062         # dispatch) doesn't run multiple compiles at once on the box (see
_LOCAL_COMPUTE_LANE).
1063         with _LOCAL_COMPUTE_LANE:
1064             out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
1065         except Exception as e: # noqa: BLE001 ? a stitch/ffmpeg failure is a clean job
FAILURE, not a crash
1066             logger.error(
1067                 "[compile] stitching failed for video_id=%s: %s", video_id, e, exc_info=True
1068             )
1069             return {
1070                 "status": "failed",
1071                 "message": f"Highlight stitching failed: {e}",
1072                 "video_id": video_id,
1073             }
1074         finally:
1075             if local_src:
1076                 storage.cleanup_acquired_local_input(
1077                     video["filepath"], local_src, getattr(cfg, "storage", None)
1078                 )
1079
1080         # #463: mirror the reel to the output bucket so /api/highlights can 307 to a signed
URL ? Cloud Run
1081         # caps a buffered response at ~32 MiB (a real reel is bigger) and the local /tmp
copy dies when the
1082         # instance recycles. A mirror-upload failure must NOT fail an otherwise-good stitch
(the endpoint
1083         # still falls back to the local file on the same instance).
1084         reel_uri = reel_object_uri(cfg, video_id, out_name)
1085         if reel_uri:
1086             try:
1087                 notify("uploading reel")
1088                 # Thread the active storage settings so the configured backend (S3
endpoint/region, GCS
1089                 # project/client) is used, not a fresh default.
1090                 storage.put(out_path, reel_uri, config=_storage_settings(cfg))
1091             except Exception as e: # noqa: BLE001 ? best-effort mirror; local stitch
already succeeded
1092                 logger.warning("[compile] could not mirror reel to %s: %s", reel_uri, e)
1093
1094         # `filepath` is server-local (not browser-reachable); also return the download URL
so the SPA needn't
1095         # hand-build it (P6 / PER_RALLY_SELECTION.md §4.3).
1096         download_url = f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
1097         notify("finished")
1098         return {
1099             "status": "success",
1100             "filepath": out_path,
1101             "download_url": download_url,
1102             "video_id": video_id,
1103         }
1104

```

14. assistant (2026-07-07T14:46:47.627Z)

Let me examine the reporting harvest (val_context consumer) and the Database.add_video invariants.

15. user (2026-07-07T14:46:49.134Z)

```

1      """Per-video observability reports for the indexer.
2
3      `obsreport` is a SOFT dependency: if it is not installed, this module degrades to a
4      no-op and the pipeline runs exactly as before. The single entrypoint
5      `emit_indexer_report(...)` is designed to be called from a `finally` block and
6      NEVER raises ? emitting a report must not be able to break a processing run.
7      """
8
9      from __future__ import annotations
10
11      from typing import Any, Dict, List, Optional
12
13      from . import run_signals # no obsreport dependency ? always importable
14
15      try: # soft dependency ? never hard-fail the pipeline on a reporting import
16          import obsreport # noqa: F401
17
18          from . import harvest
19
20          OBSREPORT_AVAILABLE = True
21      except Exception: # pragma: no cover - exercised only when obsreport is absent
22          OBSREPORT_AVAILABLE = False
23
24
25      def report_enabled(config: Optional[Dict[str, Any]]) -> bool:
26          return bool((config or {}).get("report", {}).get("enabled", True))
27
28
29      def emit_indexer_report(
30          *,
31          db,
32          video_id: str,
33          video_path: Optional[str],
34          config: Optional[Dict[str, Any]],
35          val_results: List[Any],
36          val_context: Optional[Dict[str, Any]] = None,
37          segmenter_requested: Optional[str] = None,
38          segmenter_actual: Optional[str] = None,
39          was_reingest: bool = False,
40          segmentation_ran: bool = True,
41      ) -> Optional[Dict[str, str]]:
42          """Harvest + write the report. Returns the written file paths, or None if
43          reporting is unavailable/disabled/failed. Never raises."""
44          config = config or {}
45          # Accept either a plain dict or a typed config object (e.g.
46          backend.config.AppConfig).
47          # Normalize to a plain nested dict so harvest's dict access works uniformly.
48          if hasattr(config, "model_dump"):
49              try:
50                  config = config.model_dump(mode="json")
51              except Exception:
52                  config = {}
53          # Consume run signals unconditionally (even when reporting is off) so the
54          # process-local store never accumulates entries for finished runs.
55          signals = run_signals.pop(video_id)
56          if not OBSREPORT_AVAILABLE or not report_enabled(config):
57              return None
58          try:
59              play_segments = db.query_play_segments(video_id, {}) if db is not None else []
60              candidates = db.query_candidate_segments(video_id) if db is not None else []
61              rec = harvest.record_run(
62                  video_id=video_id,
63                  video_path=video_path,
64                  config=config,
65                  val_results=val_results,

```

```

65         val_context=val_context,
66         play_segments=play_segments,
67         candidates=candidates,
68         segmenter_requested=segmenter_requested,
69         segmenter_actual=segmenter_actual,
70         was_reingest=was_reingest,
71         segmentation_ran=segmentation_ran,
72         run_signals=signals,
73     )
74     output_dir = (config.get("report") or {}).get("output_dir")
75     res = rec.write(output_dir=output_dir)
76     return res.paths or None
77 except Exception: # pragma: no cover - defensive; reporting must never break a run
78     return None
79

```

16. user (2026-07-07T14:46:49.366Z)

```

1     """Build an observability report for one indexer run.
2
3     Pure-ish harvest: given the validation results/context, the persisted segments,
4     and run metadata, populate an `obsreport.RunRecorder` with the raw signals. The
5     *footgun rules* (which turn signals into flags) live declaratively in spec.yaml,
6     so this module only records facts faithfully ? it does not decide warnings.
7
8     Imported only after `backend.reporting` confirms obsreport is installed, so it may
9     import obsreport freely.
10    """
11
12    from __future__ import annotations
13
14    import os
15    from pathlib import Path
16    from typing import Any, Dict, List, Optional
17
18    try:
19        import obsreport # soft dep; only needed for RunRecorder / load_spec at runtime
20    except ImportError:
21        obsreport = None # type: ignore[assignment]
22
23    _AI_SEGMENTERS = {"yolo_hybrid", "tracknet_hybrid", "wasb_hybrid", "gemini"}
24    _SPEC_PATH = Path(__file__).resolve().parent / "spec.yaml"
25
26
27    def load_spec() -> "obsreport.ReportSpec":
28        return obsreport.load_spec(_SPEC_PATH)
29
30
31    # --- media metadata -----
32    def video_meta_from_context(
33        val_context: Optional[Dict[str, Any]], video_path: Optional[str]
34    ) -> Dict[str, Any]:
35        """Derive VideoMeta fields from the validators' shared ffprobe_meta. fps_source
36        distinguishes a real probe from the absence of one (the fallback footgun)."""
37        meta: Dict[str, Any] = {"fps_source": "unknown"}
38        if video_path and os.path.exists(video_path):
39            try:
40                meta["size_mb"] = round(os.path.getsize(video_path) / (1024 * 1024), 1)
41            except OSError:
42                pass
43
44        ffmata = (val_context or {}).get("ffprobe_meta") or {}
45        streams = ffmata.get("streams") or []
46        fmt = ffmata.get("format") or {}
47        if streams:

```

```

48         s = streams[0]
49         w, h = s.get("width"), s.get("height")
50         meta["width"], meta["height"] = w, h
51         if w and h:
52             meta["resolution"] = f"{w}x{h}"
53         fps_str = s.get("r_frame_rate", "0/1")
54         try:
55             num, den = (int(x) for x in fps_str.split("/"))
56             if den:
57                 meta["fps"] = round(num / den, 2)
58                 meta["fps_source"] = "probed"
59         except (ValueError, ZeroDivisionError):
60             pass
61         if s.get("codec_name"):
62             meta["container"] = fmt.get("format_name") or s.get("codec_name")
63     if fmt.get("duration"):
64         try:
65             meta["duration_s"] = round(float(fmt["duration"]), 1)
66         except (ValueError, TypeError):
67             pass
68     if fmt.get("format_name"):
69         meta["container"] = fmt.get("format_name")
70
71     # Audio readiness signal (for Input Quality UX extension per review item 2).
72     # Guidelines make "record with audio on, mic unobstructed" a first-class requirement
73     # (ADR-007) because shuttle thwacks are a strong rally cue for recall.
74     # We only record presence here (cheap, from ffprobe already in context); the actual
75     # thwack onset probe (pipeline/audio/hit_detector) remains a diagnostic /
calibration
76     # tool for now.
77     audio_streams = [s for s in streams if (s or {}).get("codec_type") == "audio"]
78     meta["audio_present"] = len(audio_streams) > 0
79     return meta
80
81
82     # --- stage builders -----
83     def _validation_stage(rec: "obsreport.RunRecorder", val_results: List[Any]) -> None:
84         with rec.stage("validation") as st:
85             total = len(val_results)
86             passed = sum(1 for r in val_results if getattr(r, "passed", False))
87             st.add_metric("checks_passed", passed)
88             st.add_metric("checks_total", total)
89             failures = [r for r in val_results if not getattr(r, "passed", True)]
90             for r in failures:
91                 st.messages.append(
92                     f"FAILED {getattr(r, 'validator_name', '?')}: {getattr(r, 'message',
93 ''}})"
94                 )
95             st.status = "error" if failures else ("ok" if total else "not_run")
96
97     def _segmentation_stage(
98         rec: "obsreport.RunRecorder",
99         play_segments: List[Dict[str, Any]],
100         candidates: List[Dict[str, Any]],
101         segmenter_requested: Optional[str],
102         segmenter_actual: Optional[str],
103         config_default: Optional[str],
104         was_reingest: bool,
105         ran: bool,
106     ) -> None:
107         with rec.stage("segmentation") as st:
108             if not ran:
109                 st.status = "not_run"
110             seg_count = len(play_segments)

```

```

111     total_play = round(sum(float(s.get("duration", 0.0)) for s in play_segments), 1)
112     detector_tag = None
113     if play_segments:
114         detector_tag = (play_segments[0].get("metadata") or {}).get("detector")
115         st.add_metric("candidate_windows", len(candidates))
116         st.add_metric("segment_count", seg_count, audience="both")
117         st.add_metric("total_play_s", total_play, unit="s")
118         # "diverges" only when a segmenter actually RAN, the user did NOT explicitly
119         # pick it, and it differs from config's default (the genuine surprise case).
120         # An explicit --segmenter override or a halted run is not a divergence.
121         matches_default = True
122         if ran and segmenter_actual and config_default:
123             matches_default = segmenter_actual == config_default
124         st.details.update(
125             {
126                 "segmenter_requested": segmenter_requested,
127                 "segmenter_actual": segmenter_actual,
128                 "config_default": config_default,
129                 "segmenter_explicitly_requested": segmenter_requested is not None,
130                 "matches_config_default": matches_default,
131                 "was_reingest": was_reingest,
132                 "detector": detector_tag,
133             }
134         )
135         # Surface metadata trustworthiness at the data level (not just in a warning):
136         # the motion baseline fabricates per-segment metadata; everything else measures
137         it.
138         if ran and segmenter_actual:
139             st.details["metadata_source"] = (
140                 "synthetic/heuristic ? NOT measured"
141                 if segmenter_actual == "motion"
142                 else "measured (detector/AI)"
143             )
144
145     def _ai_stage(
146         rec: "obsreport.RunRecorder",
147         config: Dict[str, Any],
148         segmenter_actual: Optional[str],
149         play_segments: List[Dict[str, Any]],
150         ran: bool,
151         run_signals: Optional[Dict[str, Any]] = None,
152     ) -> None:
153         """Only meaningful for AI-based segmenters. Records whether the provider key was
154         present ? the signal the silent_provider_noop rule keys on ? plus any in-process
155         run signals (backend.reporting.run_signals), e.g. oversize chunk refusals."""
156         ai_based = segmenter_actual in _AI_SEGMENTERS
157         provider = config.get("ai_provider") or "gemini"
158         key_env = f"{provider.upper()}_API_KEY"
159         with rec.stage("ai_handoff") as st:
160             if not ran or not ai_based:
161                 st.status = "not_run" if not ran else "skipped"
162             st.details.update(
163                 {
164                     "ai_based": ai_based,
165                     "provider": provider if ai_based else None,
166                     "model": config.get("ai_model") if ai_based else None,
167                     "api_key_present": bool(os.environ.get(key_env)) if ai_based else None,
168                     # Chunks the provider refused as over its upload cap (no analysis ran
169                     # for
170                     # those time ranges). Detail name couples to the oversize_clip_skipped
171                     "oversize_clips_skipped": int(
172                         (run_signals or {}).get("oversize_clips_skipped", 0)
173                     )
174                 }
175             )

```

```

173             if ai_based
174             else None,
175         }
176     )
177     if ai_based:
178         st.add_metric("confirmed_segments", len(play_segments))
179
180
181 def _highlights(
182     rec: "obsreport.RunRecorder",
183     play_segments: List[Dict[str, Any]],
184     video_meta: Dict[str, Any],
185 ) -> None:
186     total = round(sum(float(s.get("duration", 0.0)) for s in play_segments), 1)
187     coverage = None
188     dur = video_meta.get("duration_s")
189     if dur:
190         coverage = round(100.0 * total / dur, 1)
191     notes = []
192     if not play_segments:
193         notes.append("No highlights were produced for this video.")
194     rec.set_highlights(
195         segment_count=len(play_segments),
196         total_highlight_s=total,
197         coverage_pct=coverage,
198         notes=notes,
199     )
200
201
202 # --- top-level -----
203 def record_run(
204     *,
205     video_id: str,
206     video_path: Optional[str],
207     config: Dict[str, Any],
208     val_results: List[Any],
209     val_context: Optional[Dict[str, Any]],
210     play_segments: List[Dict[str, Any]],
211     candidates: List[Dict[str, Any]],
212     segmenter_requested: Optional[str] = None,
213     segmenter_actual: Optional[str] = None,
214     was_reingest: bool = False,
215     segmentation_ran: bool = True,
216     tool_version: Optional[str] = None,
217     spec: Optional["obsreport.ReportSpec"] = None,
218     run_signals: Optional[Dict[str, Any]] = None,
219     **recorder_kwargs: Any,
220 ) -> "obsreport.RunRecorder":
221     """Assemble a finalized recorder for an indexer run (does NOT write)."""
222     rec = obsreport.RunRecorder(
223         "badminton-highlight-indexer",
224         tool_version=tool_version,
225         video_id=video_id,
226         video_id_kind="file_md5",
227         video_path=video_path,
228         spec=spec if spec is not None else load_spec(),
229         **recorder_kwargs,
230     )
231     vm = video_meta_from_context(val_context, video_path)
232     # If we never probed an fps but segmentation still ran, the segmenter must have
233     # used its internal fallback (30fps / 1280px) ? surface that as the footgun it is,
234     # AND show the assumed numeric value so the report reads "30 (fallback)" not "?"
235     (fallback)".
236     if vm.get("fps_source") == "unknown" and segmentation_ran:
237         vm["fps_source"] = "fallback"

```

```

237         if vm.get("fps") is None:
238             vm["fps"] = (
239                 30.0 # matches the runners' fallback (_probe_fps_width -> 30.0 / 1280)
240             )
241     rec.set_video_meta(**vm)
242
243     _validation_stage(rec, val_results)
244     config_default = (config.get("indexing") or {}).get("default_segmenter")
245     _segmentation_stage(
246         rec,
247         play_segments,
248         candidates,
249         segmenter_requested,
250         segmenter_actual,
251         config_default,
252         was_reingest,
253         segmentation_ran,
254     )
255     _ai_stage(
256         rec, config, segmenter_actual, play_segments, segmentation_ran, run_signals
257     )
258     _highlights(rec, play_segments, vm)
259
260     rec.finalize()
261     return rec
262

```

17. assistant (2026-07-07T14:47:01.450Z)

Let me read cloud_run.py to check the argv construction and verify nothing else depends on validation.

18. user (2026-07-07T14:47:03.150Z)

```

160         policy: ExecutionPolicy = DEFAULT_POLICY,
161         token: Optional[str] = None,
162         container_overrides: Optional[tuple[str, ...]] = None,
163     ) -> None:
164         self._client = run_client
165         self._job_name = job_name
166         self._region = region
167         self._project = project
168         self._storage_config = storage_config or {}
169         self._policy = policy
170         # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
171         # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
(tests).
172         self._token = token
173         self._container_overrides = (
174             _DEFAULT_CONTAINER_OVERRIDES
175             if container_overrides is None
176             else tuple(container_overrides)
177         )
178
179     def run_infer(
180         self,
181         spec: ComputeJobSpec,
182         *,
183         on_wait: "Callable[[float], None] | None" = None,
184     ) -> ComputeResult:
185         out_root = spec.out_uri.rstrip("/")
186         token = self._token or uuid.uuid4().hex
187         done_uri = f"{out_root}/_dispatch/{token}.done"
188         argv: List[str] = [

```

```

189         "infer",
190         "--video-uri",
191         spec.video_uri,
192         "--out-uri",
193         spec.out_uri,
194         "--done-uri",
195         done_uri,
196     ]
197     if spec.segmenter:
198         argv += ["--segmenter", spec.segmenter]
199     if spec.skip_validation:
200         # The control-plane already validated + only dispatches on PASS ? tell the
worker not to
201         # re-decode and re-validate (redundant, and its config may resolve different
thresholds).
202         argv.append("--skip-validation")
203     for kv in (*spec.config_overrides, *self._container_overrides):
204         argv += ["--set", kv]
205
206     execution = self._client.run_job(
207         self._job_name, argv, region=self._region, project=self._project
208     )
209     LOG.info(
210         "cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
211         self._job_name,
212         execution,
213         done_uri,
214     )
215
216     try:
217         marker = poll_until(
218             lambda: self._read_marker(done_uri),
219             self._policy,
220             label="cloud-run-infer",
221             logger=LOG,
222             # Live heartbeat (#482): each still-waiting tick reaches the caller so
223             # job.progress moves during the whole GPU run instead of freezing.
224             on_tick=on_wait,
225         )
226     except PolicyTimeout as timeout_exc:
227         marker = self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
228         video_id = str(marker.get("video_id", ""))
229         # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
230         # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
231         rc = int(marker.get("returncode", 1))
232         if not video_id:
233             LOG.warning(
234                 "cloud-run: job=%s completion marker present but empty/unreadable (no "
235                 "video_id) ? treating as failure",
236                 self._job_name,
237             )
238             rc = rc or 1
239         outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""

```

19. assistant (2026-07-07T14:47:57.438Z)

The empty-validation lens is well-guarded. Let me now investigate a rollout-skew angle: `--skip-validation` is a brand-new argparse flag, and the control-plane and worker deploy as separate artifacts. Let me check the deployment model.

20. user (2026-07-07T14:47:58.535Z)

No files found

21. user (2026-07-07T14:47:59.180Z)

No files found

22. assistant (2026-07-07T14:48:06.513Z)

The Grep searched the wrong root (cwd is a stub). Let me search the worktree explicitly.

23. user (2026-07-07T14:48:07.770Z)

Found 6 files

```
C:\Users\avidu\Projects\khealsutra-guru\bhi-wt-worker-reval\tests\test_worker.py
C:\Users\avidu\Projects\khealsutra-guru\bhi-wt-worker-reval\tests\test_api_cloud_dispatch.py
C:\Users\avidu\Projects\khealsutra-guru\bhi-wt-worker-reval\backend\orchestration.py
C:\Users\avidu\Projects\khealsutra-guru\bhi-wt-worker-reval\backend\worker\__main__.py
C:\Users\avidu\Projects\khealsutra-guru\bhi-wt-worker-reval\backend\compute\cloud_run.py
C:\Users\avidu\Projects\khealsutra-guru\bhi-wt-worker-reval\backend\compute\base.py
```

24. user (2026-07-07T14:48:08.946Z)

Found 11 files

```
C:\Users\avidu\Projects\khealsutra-guru\bhi-wt-worker-reval\deploy\gcp\provision.sh
C:\Users\avidu\Projects\khealsutra-guru\bhi-wt-worker-reval\deploy\gcp\teardown.sh
C:\Users\avidu\Projects\khealsutra-guru\bhi-wt-worker-reval\deploy\gcp\nightly_regression\setup_email.sh
C:\Users\avidu\Projects\khealsutra-guru\bhi-wt-worker-reval\deploy\gcp\ops-agent-startup.sh
C:\Users\avidu\Projects\khealsutra-guru\bhi-wt-worker-reval\deploy\gcp\nightly_regression\run_on_vm.sh
C:\Users\avidu\Projects\khealsutra-guru\bhi-wt-worker-reval\deploy\gcp\nightly_regression\launch.sh
C:\Users\avidu\Projects\khealsutra-guru\bhi-wt-worker-reval\deploy\gcp\nightly_regression\register_scheduler.sh
C:\Users\avidu\Projects\khealsutra-guru\bhi-wt-worker-reval\deploy\gcp\nightly_regression\create_nightly_bucket.sh
C:\Users\avidu\Projects\khealsutra-guru\bhi-wt-worker-reval\deploy\gcp\create_gpu_vm.sh
C:\Users\avidu\Projects\khealsutra-guru\bhi-wt-worker-reval\deploy\digitalocean\gpu_setup.sh
C:\Users\avidu\Projects\khealsutra-guru\bhi-wt-worker-reval\deploy\digitalocean\bootstrap.sh
```

25. user (2026-07-07T14:48:15.710Z)

(Bash completed with no output)

26. user (2026-07-07T14:48:21.246Z)

```
deploy/cloudrun/cloudbuild.yaml:1:# Cloud Build config for the Cloud Run GPU worker image
(COMPUTE_DECOUPLED_SERVING M7).
deploy/cloudrun/cloudbuild.yaml:8:# Override the region/repo/image with
--substitutions=_REGION=?,_REPO=?,_IMAGE=? if needed.
deploy/cloudrun/cloudbuild.yaml:13: _IMAGE: rally-worker
deploy/cloudrun/cloudbuild.yaml:19: -
'${_REGION}-docker.pkg.dev/${PROJECT_ID}/${_REPO}/${_IMAGE}:latest'
deploy/cloudrun/cloudbuild.yaml:23:images:
deploy/cloudrun/cloudbuild.yaml:24: -
'${_REGION}-docker.pkg.dev/${PROJECT_ID}/${_REPO}/${_IMAGE}:latest'
deploy/cloudrun/cloudbuild.yaml:26: machineType: E2_HIGHCPU_8 # the CUDA + micromamba image
is heavy; a bigger builder keeps it ~8?10 min
deploy/cloudrun/Dockerfile:1:# Cloud Run GPU **Job** image for the rally worker
(COMPUTE_DECOUPLED_SERVING M6/M7, ADR-012/D16).
deploy/cloudrun/Dockerfile:5:# M7 real run (the $100/?10k budget, D17); CI does NOT build this
(a CUDA image is too heavy) ? the
deploy/cloudrun/Dockerfile:17:# commercial sharing. They are NOT baked into this image; stage
them at runtime (a bucket fetch or a
deploy/cloudrun/Dockerfile:25:# so a large version skew can break decode ? the golden gate on
the real image is the check.
```

```

deploy/cloudrun/Dockerfile:94:# google.cloud`. cloud-run: this SAME image is reused as the
deployed CONTROL PLANE, which dispatches
deploy/cloudrun/Dockerfile:101:# deploy with edge-auth ON needed a hand-built derived image (the
A1 deploy's PyJWT overlay).
deploy/cloudrun/gen_service_yaml.py:8: gcloud run services replace service.yaml --region
asia-southeast1
deploy/cloudrun/gen_service_yaml.py:10:Why a MANIFEST (`services replace`) and not `gcloud run
deploy` flags: the container command is
deploy/cloudrun/gen_service_yaml.py:17:server, so one image serves every config shape
(config.json wins over the cloud-serving preset ?
deploy/cloudrun/gen_service_yaml.py:91: """Render the Knative service manifest for `gcloud
run services replace`."""
deploy/cloudrun/gen_service_yaml.py:120:         - image: {args.image}
deploy/cloudrun/gen_service_yaml.py:154:         "--image",
deploy/cloudrun/gen_service_yaml.py:156:         help="Container image; default = the project's
rally-worker:latest (post-#470 it "
deploy/cloudrun/gen_service_yaml.py:157:         "carries cloud-run + auth, so the worker image
IS the control-plane image).",
deploy/cloudrun/gen_service_yaml.py:192:         if args.image is None:
deploy/cloudrun/gen_service_yaml.py:193:             args.image =
f"{args.region}-docker.pkg.dev/{args.project}/rally/rally-worker:latest"
deploy/cloudrun/post_deploy_smoke.ps1:4: the key log lines + output artifacts. Run after
cutting the Job over to a new image (and/or
deploy/cloudrun/post_deploy_smoke.ps1:8: `gcloud run jobs execute --args=...` with gs:// URIs
mangles under Git Bash (MSYS path
deploy/cloudrun/post_deploy_smoke.ps1:49:gcloud run jobs execute $Job --region $Region --project
$Project --async --args=$argstr 2>&1 | Select-Object -Last 2
deploy/cloudrun/post_deploy_smoke.ps1:51:$exec = gcloud run jobs executions list --job $Job
--region $Region --project $Project --limit 1 --format="value(metadata.name)" 2>&1 | Select-
Object -Last 1
deploy/cloudrun/post_deploy_smoke.ps1:57: $vals = gcloud run jobs executions describe $exec
--region $Region --project $Project --format="value(status.succeededCount,status.failedCount,sta
tus.runningCount,status.completionTime)" 2>&1 | Select-Object -Last 1
deploy/cloudrun/post_deploy_smoke.ps1:58: $done = gcloud run jobs executions describe $exec
--region $Region --project $Project --format="value(status.completionTime)" 2>&1 | Select-Object
-Last 1
deploy/cloudrun/README.md:4:dispatches a Cloud Run **Job** execution running this image's worker
on an **L4***, then **bucket-polls**
deploy/cloudrun/README.md:7:> **Status:** the dispatch shim + image are **landed + mock-tested**
(M6). The **real build + push +
deploy/cloudrun/README.md:9:> cost first, **delete every resource after** ([[gcp-vm-always-
delete]]). CI does **not** build this image.
deploy/cloudrun/README.md:17:## 1. Build + push the image (Artifact Registry)
deploy/cloudrun/README.md:26:*(A CUDA + micromamba image is large; the first build is ~8?10 min.
Budget-watch per D17.)*
deploy/cloudrun/README.md:30:mounted read-only via GCS FUSE (no weights in the image ? WASB-C3;
no init-fetch step needed).
deploy/cloudrun/README.md:32:gcloud run jobs create rally-worker \
deploy/cloudrun/README.md:33:  --image $REGION-docker.pkg.dev/$PROJECT/$REPO/$IMG:latest \
deploy/cloudrun/README.md:50:The image ships the **py3.10 + torch 2.6.0+cu124 +
PyNvVideoCodec==2.1.0** WASB env with
deploy/cloudrun/README.md:71:gcloud run jobs update rally-worker --region $REGION --update-env-
vars WASB_DECODE_BACKEND=nvdec_resident
deploy/cloudrun/README.md:72:gcloud run jobs update rally-worker --region $REGION --remove-env-
vars WASB_DECODE_BACKEND # ? forwarded/preset default
deploy/cloudrun/README.md:73:# full revert: pin the prior image by digest
deploy/cloudrun/README.md:74:gcloud run jobs update rally-worker --region $REGION --image
$REGION-docker.pkg.dev/$PROJECT/$REPO/rally-worker@sha256:<prev>
deploy/cloudrun/README.md:76:? Forwarding `decode_backend=nvdec_resident` requires a worker
image that understands
deploy/cloudrun/README.md:77:`--decode-backend` (? #507). Roll the **worker image and the
control plane together** ? a new
deploy/cloudrun/README.md:78:control plane forwarding nvdec to an old worker image argparse-
fails (rc 2).
deploy/cloudrun/README.md:83:./deploy/cloudrun/post_deploy_smoke.ps1 -Video gs://khelsutra-

```

```

rally-corpus/videos/mahadevpura_smoke_180s.mp4 -Decode cpu    # image sanity (fast)
deploy/cloudrun/README.md:116:gcloud run services replace service.yaml --region $REGION    # from
PowerShell on Windows
deploy/cloudrun/README.md:122:gcloud run deploy rally-control-plane --image <image> --region
$REGION \
deploy/cloudrun/README.md:134:(M9) before inviting testers. The image ships the `auth` extra
(PyJWT[crypto]), so flipping
deploy/cloudrun/README.md:135:`security.edge_auth_enabled` on needs no derived image ? the M9
Cf-Access JWT verify works out of
deploy/cloudrun/README.md:154:- libx264 encode (parity, D16) + NVDEC decode-only; the image
keeps ffmpeg's default libx264.
deploy/cloudrun/README.md:172:- **First execution after an image push** spends minutes in
"Container image import in progress"
deploy/cloudrun/README.md:174: later executions reuse the imported image.
deploy/digitalocean/gpu_setup.sh:13:# PyNvVideoCodec needs (compute-only driver images ship
neither).
deploy/digitalocean/gpu_setup.sh:15:# Requires an NVIDIA GPU + driver (the DO `gpu-*-base` image
ships it).
deploy/digitalocean/gpu_setup.sh:36:# Compute-only driver images (GCE DLVM, DO gpu-*-base) ship
neither libnvcuvid.so.1 nor
deploy/digitalocean/gpu_setup.sh:61:# box. (Observed on a GCP Deep-Learning-VM image,
2026-06-20.)
deploy/digitalocean/gpu_setup.sh:100:assert ok, "CUDA not available in the wasb env ? check the
GPU driver / base image"
deploy/digitalocean/README.md:35:**Secret hygiene:** never bake any of these into the repo or a
Docker image. Put them in the box's
deploy/gcp/create_gpu_vm.sh:5:# metrics), and uses a driver-ready Deep-Learning-VM image
(nvidia-smi works immediately).

```

27. assistant (2026-07-07T14:49:48.794Z)

I have what I need. Let me confirm the README's documented rollout discipline to cite it precisely.

28. user (2026-07-07T14:49:50.438Z)

```

60      **How the knob reaches the worker.** The worker Job runs with NO `DEPLOYMENT_PROFILE`,
so it does
61      **not** apply the cloud-serving preset itself. The control plane (which *does* have the
profile)
62      resolves the config and **forwards**
`indexing.wasb.{decode_backend,batch_size,prefetch_batches}`
63      to the worker as `--set` overrides (`orchestration.py`, alongside `skip_ai_handoff`). So
the
64      **preset is the single source of truth** ? changing `presets.py` + redeploying the
control plane
65      changes serving; no per-Job knob needed.
66
67      **Override / rollback lever (no redeploy).** The native runner reads
`WASB_DECODE_BACKEND` FIRST,
68      so a Job env var wins over the forwarded `--set`:
69      ```bash
70      # force nvdec (or pin cpu) on the Job regardless of the forwarded config; instant
rollback = remove it
71      gcloud run jobs update rally-worker --region $REGION --update-env-vars
WASB_DECODE_BACKEND=nvdec_resident
72      gcloud run jobs update rally-worker --region $REGION --remove-env-vars
WASB_DECODE_BACKEND # ? forwarded/preset default
73      # full revert: pin the prior image by digest
74      gcloud run jobs update rally-worker --region $REGION --image $REGION-
docker.pkg.dev/$PROJECT/$REPO/rally-worker@sha256:<prev>
75      ```
76      ? Forwarding `decode_backend=nvdec_resident` requires a worker image that understands
77      `--decode-backend` (? #507). Roll the **worker image and the control plane together** ?
a new

```

```

78     control plane forwarding nvdec to an old worker image argparse-fails (rc 2).
79
80     **Post-deploy smoke.** After a cutover, confirm the real L4 path with
81     `deploy/cloudrun/post_deploy_smoke.ps1` (run from PowerShell ? see $Gotchas on MSYS arg-
mangling):
82     ```powershell
83     ./deploy/cloudrun/post_deploy_smoke.ps1 -Video gs://khelsutra-rally-
corpus/videos/mahadevpura_smoke_180s.mp4 -Decode cpu    # image sanity (fast)
84     ./deploy/cloudrun/post_deploy_smoke.ps1 -Video gs://khelsutra-rally-
corpus/videos/GX010128.MP4      -Decode env    # nvdec via preset/env (hard clip)
85     ```
86
87     ## 3. Wire the dispatcher (control plane)
88     The control plane runs with the **cloud-serving** profile (`DEPLOYMENT_PROFILE=cloud-
serving` ?
89     `compute_target=cloud_run`). `get_compute_backend(config)` then builds `CloudRunCompute`
from env:
90     ```bash
91     export DEPLOYMENT_PROFILE=cloud-serving
92     export CLOUD_RUN_JOB=rally-worker CLOUD_RUN_REGION=$REGION
GOOGLE_CLOUD_PROJECT=$PROJECT
93     # A `worker infer` (or the API job path) with a gs:// video now DISPATCHES to the Job
instead of
94     # running in-process; the container runs the SAME worker INLINE (compute_target forced
local_gpu).
95     python -m backend.worker infer --video-uri gs://<bucket>/videos/clip.mp4 --out-uri
gs://<bucket> --segmenter wasb_hybrid
96     ```
97     **The AI handoff needs its key IN THE JOB env** (`GEMINI_API_KEY` ? mount a Secret
Manager secret
98     on the job), or run fully-local with `--set indexing.skip_ai_handoff=true` (candidate
windows
99     emitted as rallies ? the eval-parity mode). With neither, the segmenter used to bail to
a silent

```

29. assistant (2026-07-07T14:50:21.182Z)

I've traced every downstream consumer of the empty validation state and found the empty-validation lens itself is well-guarded. But I found a real rollout defect. Let me summarize my analysis before reporting.

Empty-validation lens ? traced, no breakage found

```

- **Worker `report.json` / `intervals.csv`**: `_write_report_json` only emits
`video_id/status/segment_count/video_uri/segments`; `_write_intervals_csv` only emits rally
rows. Neither ever carried `validation_results` or `ffprobe_meta`, so
`val_results=[]`/`val_context={}` don't change or malform these artifacts.
- **Control-plane ingest** (`_ingest_remote_segments`, `probe_process_cache`,
`cached_process_result`, `_read_bucket_json`): read only `report["status"]`,
`report["video_uri"]`, and interval CSV columns. No path reads validation/ffprobe from the
worker output.
- **`db.add_video("processed", [])` on the worker**: the worker DB lives in ephemeral scratch
and is never pushed; the control-plane DB is written with the CP's own non-empty `val_results`.
No invariant consumes worker validation.
- **`emit_indexer_report` with empty `val_context`**: fully wrapped, never raises; the worker's
obsreport is written to local scratch and never pushed/ingested. With empty context it labels
fps as "fallback 30", but nothing downstream reads it.
- **`0 validators ran as failure`**: `failures = [r for r in val_results if not r.passed]` ?
`[]` ? treated as pass. Clean.

```

The real defect: rollout skew from an unconditional new argparse flag

`\_maybe\_dispatch\_remote` sets `skip\_validation=True` on **every** cloud dispatch, and
`cloud\_run.py` emits it as the **hard** argparse flag `--skip-validation`. The control-plane

service and the worker Cloud Run Job are the same image but are ****separate deploy targets**** rolled by separate `gcloud` commands. The repo's own runbook already documents this exact hazard class for `decode\_backend` (README §76-78: "a new control plane forwarding ? to an old worker image argparse-fails (rc 2). Roll the worker image and the control plane together") ? but that note was not extended, and unlike `decode\_backend` this flag is forwarded unconditionally with no preset gate and no env-var escape hatch, so the blast radius is 100% of cloud dispatches.

30. user (2026-07-07T14:50:32.765Z)

Structured output provided successfully